HTL Leonding
next level

Harness Engineering

Building the machine that checks the agent's work

HTL Leonding

October 2026

Table of contents

Preface 4

1 Introduction 5
1.1 From coder to harness engineer 6
1.2 Your toolkit 8

2 Workshop 10
2.1 Start the container 11
2.2 Meet pi 12
2.3 Your run will differ 14
2.4 Git is your undo button 15

3 Context 16
3.1 The cutoff problem 17
3.2 AGENTS.md 19
3.3 The first prompt 21
3.4 Review the result 22

4 The quality gate 24
4.1 Analyzers and a gate script 25
4.2 Own the configuration 27
4.3 Automate the gate 29
4.4 Close the loop 32
4.5 Tell the agent 35

5 The environment 37
5.1 Persistence 38
5.2 The harness pays off 41
5.3 Integration tests 43
5.4 Break the environment 45

6 A second toolchain 47
6.1 More sources of truth 48
6.2 Route the docs 50
6.3 The Angular app 53
6.4 The calculations page 56
6.5 The values page 57
6.6 Playwright and tiers 58

7 Skills 60
7.1 Write a skill 61

8 The last stage 63
8.1 Lint the frontend 64
8.2 Check, never fix 66
8.3 Count the cost 68

Appendices **70**
A Troubleshooting 70

Preface

This is a hands-on course about the machine around the coding agent. You build that machine step by step. While you build it, the agent builds a full-stack application for you: a .NET 10 backend, an Angular frontend, and tests at three levels. You will type very little of that application yourself. Instead, you type the rules, the checks, and the feedback loops that decide whether the agent's output is acceptable.

Every command and every prompt in this book ran end-to-end against the same agent and the same model that run in your container. Nothing here is a sketch of what should work. If something behaves differently on your machine, write it down and bring it to class.

The book cannot promise that your agent writes the same code as the examples in these pages. Those outputs are examples, not a guarantee. The chapter "Your run will differ" explains why: it's a property of the tool, not a flaw in the book.

Part 1

Introduction

1.1 From coder to harness engineer

1.1.1 You type a prompt, and then you hope

You have all used a coding agent by now. You type a prompt and the agent writes code. Then you hope. Maybe you skim the diff. Maybe you start the app once and click around. Some days you just commit it.

Nothing in that loop tells you whether the code works. The agent is not the problem here. Coding agents write reasonable code a lot of the time. The trouble is that "a lot of the time" is a statement about odds. You never see which of today's changes fell on the wrong side of those odds.

Hoping does not scale. Reading every line by hand does not scale either. The agent produces changes faster than you can read them, and after the fourth file, you stop paying attention.

1.1.2 What your job is now

So stop thinking of yourself as the person who writes the code. You are a harness engineer now.

A harness engineer sets the guardrails and the quality gates. You decide the architecture and write the rules that everybody follows. You also decide what "done" means before anyone starts. Then you coordinate a team of agents the way a team lead coordinates people. The team members work in parallel and never get tired. But they also never tell you when they are unsure. They do the typing.

That sounds like a demotion until you look at what is left over. Every hard part of software is still yours. You decide what to build, choose the structure it lives in, define correct behavior, and prove that the thing in front of you actually has it. Only the mechanical part moved.

1.1.3 Agentic engineers, not vibe coders

"Vibe coding" is the popular name for the other way of working. You describe roughly what you want and take whatever comes back. Then you keep prompting until the app looks like it works. It is fun for a weekend demo. It falls apart the first time somebody has to change the result, because nobody, including you, can say what the code is supposed to do.

You are an agentic engineer instead: same tools, different starting point. You know what you want before you type, and you can put it into words precise enough for an agent to act on. Over a single day you move between four familiar roles.

- Coordinator, splitting work into pieces an agent can finish in one go.
- Reviewer, reading the diff and the gate output and deciding what ships.
- Software architect, fixing the structure so the agent has somewhere to put things.
- Product owner, saying what the software has to do and how you will know.

None of those roles is new. What changed is the fifth role, the one where you sit and type the implementation. That role has largely gone away.

1.1.4 Probabilistic models, deterministic tools

A large language model, or LLM, samples an answer instead of computing one. Ask the same question twice, and you get two different files. Both are plausible, but sometimes only one of them is correct. That is not a bug you can configure away. Sampling is how the model works.

Two habits follow from that, and this whole course is built on them.

Be specific wherever you care about the specifics. Every detail you leave out is a detail the model picks for you. It picks a fresh one every run. If the project has to use one HTTP client, say so in writing. If test files belong in a certain folder, say so. Vagueness is a coin flip, not flexibility.

Put deterministic tools in the path. A formatter has exactly one correct output for a given file. An analyzer either finds the rule violation or it does not. A test suite passes or fails. It gives you an exit code you

can act on. These tools never negotiate, and they behave the same way every time. They also do not care how confident the agent sounded. The model supplies the variety. The tools cut away the part of that variety you refuse to accept.

1.1.5 What a harness is

Harness engineering is the discipline of building the environment around the agent. It makes good output likely and bad output impossible to miss. The agent is the engine, and the harness is everything else.

Harness element	What it does
Context	Which documents and rules the agent sees (`AGENTS.md`, skills)
Tools	What the agent can reach: a documentation CLI, the package manager, the database
Gates	Deterministic checks the code has to survive: formatter, analyzer, tests
Feedback loops	How a failed gate gets back to the agent by itself (a `pi` extension)

The harness has one rule: never trust the agent. You verify its output, and then you automate the verification. A check you have to remember to run is a check you will skip on the day you are in a hurry.

1.1.6 The three mechanisms

Three mechanisms carry most of the harness you are about to build. Each one gets a full part of the course later, so one sentence each for now.

AGENTS.md. A markdown file in the project root. The agent reads it before it starts working. It holds the rules, the folder layout, and the commands that are true for this project and no other.

Skills. Folders of instructions. The agent pulls them in only when the task matches. That keeps a long procedure available without paying for it in every single request.

Extensions. Code the harness runs by itself at fixed points in a session, for example when the agent finishes a run. A failing gate then turns into something the agent has to deal with, not something you happen to notice.

1.1.7 The application you build

Everything in this course is built around one deliberately dull CRUD application. CRUD means the app does little more than create, read, update, and delete records. That is on purpose. When the domain is boring, all the interesting difficulty sits in the harness. That is the part you are here to learn.

- **Backend.** .NET 10 with ASP.NET Core Minimal APIs, EF Core, and PostgreSQL, with migrations applied on startup.
- **Frontend.** Angular. It reaches the backend through one hand-written API service. That service reads with the `httpResource` API.
- **Tests.** xUnit for the backend and Vitest for the frontend. Playwright drives the end-to-end tier.

Six steps build that up, in order: the backend skeleton, the first deterministic gate, and a database with integration tests and a gate for the migrations it generates. Then comes the Angular frontend with its own toolchain, a skill that teaches the harness a procedure, and linting as the final stage of the gate.

The next chapter introduces the two pieces of the setup you do not build yourself: the coding agent and the model behind it.

1.2 Your toolkit

1.2.1 The pi agent

The coding agent for this course is `pi`, an open source agent you run in a terminal. It is deliberately minimal. Out of the box it reads and writes files, edits them, and runs shell commands. That is close to the whole list. Anything beyond that, you add yourself through skills, extensions, and configuration.

That minimalism is exactly why the course uses `pi`. In a bigger agent, half the harness is already there, hidden behind a settings screen. You never find out what it does or why. With `pi`, you add every rule the agent follows and every check it has to survive yourself. All of it lives in a file you can open.

1.2.2 The model: GPT-6 Luna through Novedu

Behind `pi` sits the model that does the actual thinking. In this course, that model is GPT-6 Luna from OpenAI. Luna is the fast, low-cost model of the GPT-6 series, one step below GPT-6 Sol. Your school's Novedu platform passes every request from `pi` on to OpenRouter, a service that sells access to models from many vendors through one account.

Be clear about the trade-off. Out in industry, engineers run stronger models inside far more capable harnesses. The models include Opus, Fable, GPT-6 Sol, DeepSeek, and Kimi. The harnesses include Claude Code, OpenAI Codex, and Cursor. Those combinations take on work packages far bigger than the steps in this course: a refactoring across 40 files, a migration to a new framework version, a bug report with only a stack trace. When you judge what agentic engineering can do, judge it by those, not by the setup in your container.

1.2.3 Why a low-cost model is good for learning

Your key costs you nothing. Every request still costs your school money, because OpenRouter bills the school for each token Luna reads and each token it writes, its thinking included.

Cut the work into small steps anyway. A task cut into pieces the agent can finish in one turn is easier to check, and cutting work into those pieces is the skill this course trains. A capable model lets you skip that practice. It absorbs a sloppy prompt, and you never find out what the prompt was missing.

Mistakes are cheap. Luna is the low-cost model of its series, so a failed run costs the school little and costs you nothing. Break things on purpose and watch the gate turn red. Then you run it again. Much of this course is exactly that.

1.2.4 What you need on your machine

Docker with Compose. On Windows and macOS, install Docker Desktop, which brings Compose with it. On Linux, install the Docker engine plus the Compose plugin from your distribution or from Docker's own repository.

Visual Studio Code (VS Code). You attach it to the running container and work inside that container's terminal. Nothing in this course installs .NET, Node.js, or PostgreSQL on your own machine.

You have learned about Docker and VS Code fundamentals in other courses. If you struggle, ask your teacher or an AI. Figuring out how to work with the course material is part of the exercise.

1.2.5 The project folder from your teacher

Your teacher hands you a folder with four files and one subfolder in it. You do not have to write any of them.

- `compose.yaml` defines the two services you work with, the development container and a PostgreSQL database.
- `Dockerfile` builds the development container, with .NET 10, Node.js, the Angular CLI, `git`, the PostgreSQL client, and `pi` already installed.

- `models.json` tells `pi` which model to talk to, where that model is hosted, and how much room it gets. The chapter "Meet pi" reads those numbers with you.
- `auth.json.example` shows the shape of the credential file, with a placeholder where your own key goes.
- `skill-source/` holds the custom skills for this course. The container sees it read-only as `~/skill-source`, and the chapter "The cutoff problem" installs the skills from there into your project.

`pi` reads `auth.json`, not the example. That file is missing on purpose, because the key in it is yours alone. The next section gets you that key and turns the example into the real file.

1.2.6 Pick up your key

The model endpoint is a Novedu coding activity, and you fetch your own key from that activity's page.

Your coding agent

Sign in with your school account and the page gives you a key of your own. That key is what your coding tool needs. The code below opens the page and nothing else.
Your teacher sees that you asked for a key, never what you ask the agent.

▶ Get your coding key on app.novedu.at

Scan the code, or type this address into your browser:

`https://app.novedu.at/yasoilf2wr`

The page also names a server address and a model, and you already have both in the `models.json` from your teacher. So the key is the only thing you take away from it. In the folder your teacher gave you, copy the example file:

```
cp auth.json.example auth.json
```

Then open `auth.json` and paste the key in place of the placeholder:

```
{
  "novedu": {
    "type": "api_key",
    "key": "nvk-your-own-key-goes-here"
  }
}
```

Every key starts with `nvk-`, so a value in a different shape is the wrong string.

The key is yours

Treat the key like a password. Do not post it anywhere, and never commit `auth.json` to a repository you push. Nobody else in the class has your key, so a key that turns up somewhere else came from you. Opening the page a second time hands you the same key back, so there is nothing to lose and nothing to rotate.

If you are not a student with access to Novedu, you can still take the course. Point `pi` at GPT-6 Luna through your own OpenRouter account, or at any other model you have access to.

The next part walks you through starting the container and attaching VS Code. Then you run `pi` for the first time.

Part 2

Workshop

2.1 Start the container

Everything in this course runs inside a Docker container. Your own machine stays clean. Everyone in the room gets the same .NET SDK and Node.js, plus the same coding agent. When something breaks, it breaks the same way for everyone. That is what makes the exercises in this book checkable.

2.1.1 Start the two services

Open a terminal on your own machine and go to the folder that holds `compose.yaml`. Make sure the `auth.json` you wrote in the toolkit chapter sits in that folder too. Docker mounts it into the container without checking that it exists. When it is missing, Docker creates an empty directory under that name, and `pi` later fails with a confusing configuration error instead of a plain "key missing".

Then run:

```
docker compose up -d --build
```

`-d` puts the containers in the background. `--build` builds the image the first time and rebuilds it whenever the `Dockerfile` changes. That first build downloads the .NET SDK, Node.js, and the agent, so it takes a few minutes. Every later start takes seconds.

Two services come up.

dev. Ubuntu with .NET 10, Node.js LTS, the Angular CLI, the `pi` coding agent, `git`, `gh`, and the PostgreSQL client. This is the machine you work on for the rest of the book.

db. PostgreSQL 18, reachable from `dev` under the host name `db`. You do not touch it until the part about the environment. It only has to be running.

Check that both containers are actually up:

```
docker compose ps
```

You want `dev` running and `db` healthy. The `db` service has a health check in `compose.yaml`, and `dev` waits for it. So when `dev` refuses to start, look at `db` first.

> **Read the Dockerfile**
>
> The `Dockerfile` lists every tool the container installs. You can read it yourself or ask an AI to explain it. Nothing in the file is secret, so a free, public AI service is fine.

2.1.2 Attach VS Code

1. Install two VS Code extensions. **Container Tools** (`ms-azuretools.vscode-containers`) adds the Containers view. **Dev Containers** (`ms-vscode-remote.remote-containers`) attaches VS Code to the container.
2. Open the Containers view, right-click the running `dev` container, and choose **Attach Visual Studio Code**. A new window opens, connected to the container.
3. In that window, open a terminal. You are `root` inside the container now. You type every later command in this book there, unless a step says otherwise.

> **Do not use docker exec**
>
> `docker exec` from your host gives you a shell, and nothing else. There is no editor on the container's files and no debugger. VS Code extensions do not work in a plain shell. Every command in this book assumes the attached VS Code terminal, so attach once and stay there. You can still experiment with `docker exec` if you want. If you break things, run `docker compose down` and `docker compose up -d` to get fresh containers. Leave the `-v` off, because that flag also throws away the database and the folder you build the project in.

What you have now is an editor, a terminal, and a debugger all pointed at the container's filesystem. When the agent writes a file, you open it in the same window. Nothing gets copied between two machines.

2.2 Meet pi

`pi` is the coding agent you drive for the rest of the book. The toolkit chapter explains why it ships with so little. This chapter starts it and checks that it reaches the model.

2.2.1 Start pi for the first time

In the terminal of the attached VS Code window:

```
mkdir -p ~/work && cd ~/work
pi
```

`~/work` is the folder you build the whole project in. Start `pi` from inside it, because the agent works in the folder you launch it from.

2.2.2 Check the credential

`pi` needs two files to talk to the model. `models.json` says where the Novedu endpoint is and which model to ask for. `auth.json` holds your personal key. Both sit next to `compose.yaml` on your machine, and the compose file mounts them into the container. Because the key lives on your machine, it survives `docker compose down` and every rebuild.

When `pi` reports an authentication error instead of answering, open the `auth.json` next to `compose.yaml`. Check the JSON shape against the toolkit chapter, and check that the value starts with `nvk-`. An activity code pasted in there gets you the same error and no hint about why. After you change the file, restart the container, because Docker mounts files at startup. A key that starts with `nvk-` and still gets rejected means your teacher has closed the activity.

Now type a message and press Enter. Anything will do. You are checking one thing: whether a reply comes back at all.

> ⚠ When pi just sits there
>
> The model runs on a shared endpoint, and a request sometimes never comes back. There is no error and no retry. The session hangs before it prints anything. That happens on roughly one run in two.
> If nothing has shown up after about three minutes, press `Escape` (or `Ctrl+C`) and send the same prompt again. It is not your prompt and it is not your machine. A retry almost always works.
> You can spot a stall instead of slow work by checking whether anything is happening at all. A working agent prints tool calls as it goes. A stalled one has printed nothing since you hit Enter.
> The failure also shows up a second way: sometimes the request does come back, after a minute or two, as an empty reply. No error, no text, the prompt line just returns. Treat it the same way as a hang. Send the same prompt again.

2.2.3 The two numbers in models.json

Open `models.json` next to `compose.yaml` and read the model entry. Two of its properties decide how much room the agent gets.

`contextWindow` is how much the model can read in one request, counted in tokens. It covers the whole session so far, including every file the agent opened along the way. Here it is set to 272000.

`maxTokens` caps a single answer, and it is set to 128000. That is the most GPT-6 Luna writes in one answer, so you will not run into it by accident.

`contextWindow` sits far below the 1,050,000 tokens GPT-6 Luna can read, and that is a cost decision. Every turn sends the whole session to the model again, and OpenRouter charges more per token once a request is longer than 272,000 tokens. `pi` compacts a session, summarizing the older messages into a shorter one, as soon as the session gets within 16384 tokens of `contextWindow`. With the window at 272000, compaction starts before the session reaches the higher price. A single turn that reads several large files can still carry one request past it.

That matters when a turn goes wrong. An agent that has forgotten an instruction you gave five prompts ago has filled its context window, and the compaction summary left that instruction out. Look at what filled the window, not at the numbers in this file. Managing that budget starts in the part about context.

2.2.4 Commands and keys worth knowing

Command or key	What it does
`pi`	Start an interactive session in the current folder
`pi -c`	Continue the previous session
`pi -p "prompt"`	Run one prompt without the interactive interface, for scripting; this book never uses it
`/help`	List the slash commands
`/new`	Start a new session with an empty context window
`/compact`	Summarize the conversation so far into a shorter one
`/resume`	Pick an earlier session and continue it
`!<command>`	Run a shell command yourself, without leaving `pi`
`Escape`	Abort the current turn
`Ctrl+D`	Exit `pi` when the input line is empty

`!<command>` is the one people forget. You stay in the session and run `ls` or `git diff` yourself. You see the real output instead of asking the agent what it thinks is in the folder.

2.3 Your run will differ

Read this chapter once now. Remember it every time an output block in this book does not match your screen.

2.3.1 The agent picks the names

The agent decides most names. Two runs of the same prompt produce different class names, different file names, sometimes a different test runner. Where a name has to be stable, like the project layout and the gate script, this book pins it in the harness. You will see exactly how that works.

So sooner or later, a command fails with "no such file." Sometimes an output block does not match what is on your screen. Three rules cover that.

2.3.2 Output blocks are "something like this"

Every output block in this book is "something like this", never a promise. The strings that are quoted are quoted for a reason. Later in the course, you build a script whose last line is `QUALITY GATE: GREEN` or `QUALITY GATE: RED`. Both you and the harness read that exact text at the end of the log. Those words are stable, and they matter. The number of tests printed in the same log does not matter.

2.3.3 Substitute your own paths

When the book says `dotnet build backend/Calculator.Api/Calculator.Api.csproj` and your project sits somewhere else, use your path. Find the real one with `ls` and `git status` before you decide something is broken. `grep -rn` searches inside files when you need it.

2.3.4 A verdict that disagrees is a finding

A verdict that disagrees with the book is a finding. A transcript that disagrees is noise. If the book says the gate turns red and yours turns red, the exercise worked. It does not matter how differently your log is worded. If the book says the gate turns red and yours stays green, stop and investigate. That is a real difference. It usually means a check is not wired up the way you think.

The book is not being vague. You are working with a nondeterministic tool, so you check behavior, not appearances.

2.4 Git is your undo button

An agent run can go wrong. The first one is the most likely to. So build the safety net before you need it, not after.

2.4.1 Initialize the repository

```
cd ~/work
git init
git config user.name "Your Name"
git config user.email "you@example.com"
```

Set the identity too. A fresh container has none, so the agent's first commit fails with `Author identity unknown` and costs you a turn.

Do this now, even though the first prompt in the context part asks the agent to run `git init` as well. That happens at the end of the run, too late to rescue it, and running `git init` twice is harmless.

2.4.2 Commit before, read after

From here on, commit before every prompt and read what changed after every prompt. Use the git CLI, the VS Code Git UI, or any other Git tool you like.

A clean `git status` before a prompt means everything you see afterwards was written by that one run. The `git diff` after it is the review. It is the only place where you find out what the agent really did, not just what it says it did in its summary.

2.4.3 Throwing a bad run away

To throw a bad run away you need both of these:

```
git checkout .      # undo changes to files git already knows about
git clean -fd       # delete the new files the agent created
```

`git checkout .` alone is not enough. People often get this wrong. An agent run mostly creates files, and `git checkout` does not touch anything untracked.

This is throwaway work inside the container, and nobody expects you to push it anywhere. Git here is a safety net and a way to read what changed.

> **Where your work lives**
>
> `~/work` is not on your own machine, and it does not live inside the container either. `compose.yaml` backs that path with a named volume called `work_data`, which Docker keeps outside both. Stopping the containers with `docker compose down` leaves it alone, and so does rebuilding the image with `docker compose up -d --build`.
>
> One command does delete it. `docker compose down -v` removes the named volumes along with the containers, and that takes `~/work` and the database with it.
>
> To keep a copy on your own machine, copy the folder out of the container. This one runs in a terminal on your host, not in the attached VS Code window:
>
> ```
> docker cp harness-workshop-dev-1:/root/work ./my-harness-work
> ```
>
> `compose.yaml` pins the project name, so the container is called `harness-workshop-dev-1` on every machine in the room.

Part 3

Context

3.1 The cutoff problem

A model's knowledge stops at its training cutoff. .NET 10, C# 14, EF Core 10, and the current Angular CLI are all newer than that cutoff, or barely inside it. Ask an agent for "the latest .NET" and you get outdated code that looks right: `.sln` files instead of `.slnx`, classic extension methods where C# 14 has extension blocks, and MVC controllers instead of Minimal APIs.

Outdated code is not a prompt problem. Rewording the prompt does not add a fact the model doesn't have. The fix is to give the agent a way to look things up.

`pi` in this container can already search the web and fetch pages through a web access extension, and you will sometimes see it do that. A general web search lands wherever the search engine points, current or not. This chapter adds a source you choose: the official documentation, queried directly. Later, `AGENTS.md` tells `pi` which source to use for which question. That works because each source is a skill with a name, and `pi` loads a skill by name.

3.1.1 Install the microsoft-docs skill

A skill is a small package of instructions that the agent loads on demand. The `microsoft-docs` skill teaches `pi` how to query the official Microsoft Learn documentation.

```
cd ~/work
npx skills add https://github.com/microsoft/skills --skill microsoft-docs
```

The command stops and waits for you. It asks whether to install the skill for this project or globally. Choose Project. If you ever want to skip that question, `-y` picks the defaults.

Two things appear in `~/work` afterward:

```
~/work/.agents/skills/microsoft-docs/SKILL.md
skills-lock.json
```

Open `SKILL.md` and read the `compatibility:` line. The skill prefers the Microsoft Learn MCP server. MCP is the Model Context Protocol. It falls back to the `mslearn` command line tool. `pi` has no built-in MCP support, so in this container the `mslearn` command is the one that works.

Run one search yourself before you let `pi` use the skill:

```
npx @microsoft/learn-cli search "what is new in EF Core 10"
```

What comes back is exactly what `pi` gets to read. If this command fails on your machine, `pi` has nothing to look things up with. Everything that follows falls back to guessing.

3.1.2 Install the backend skeleton skills

The `microsoft-docs` skill tells `pi` where to find the current API. It says nothing about how your project should look: where the solution goes, what the projects are called, how an endpoint file is built. Those are your team's decisions, and no public skill knows them. A custom skill is a skill written for one team or one course.

A full ASP.NET Core backend would get a service layer and a persistence project. The calculator has no business logic worth a service, so for now the backend stays one API project plus its tests. That keeps every agent turn small enough to watch what the harness catches. The service layer and the persistence project arrive with the database, in the chapter "Persistence".

The custom skills come from `skill-source/` in the folder your teacher gave you, which the container sees read-only as `~/skill-source`. It holds one folder per stage, and `pi` never reads it. The agent reads only the copies the install puts into `~/work/.agents/skills/`. `backend-skeleton` is the stage you need now.

Install every skill in the `backend-skeleton` folder into the project:

```
cd ~/work
npx skills add ~/skill-source/backend-skeleton -y
```

`-y` skips the scope question. Since `~/work` is a Git repository, the command picks Project on its own. List the skills afterward:

```
ls .agents/skills/
```

Next to `microsoft-docs`, two new folders appear. Each holds a `SKILL.md` you can read like any other Markdown file. One covers C# style. The other covers Minimal API endpoints, and its examples write the application name as `{App}`. The skills are written for any application, so they cannot know that yours is a calculator. `AGENTS.md` supplies that name in the next step.

`skills-lock.json` now lists three skills. The two new entries say `"sourceType": "local"`, because they came from a folder rather than from GitHub.

> **Trust the project first**
>
> Project-local skills load only after you trust the project. The first time you start `pi` in `~/work`, it asks whether you trust the project. Answer yes. To confirm it worked, start a `pi` session and ask `pi` to list its skills. `microsoft-docs` and the two backend skeleton skills have to appear in the answer. You can also run `/skills` and check the list.

3.1.3 How a skill loads

Open `.agents/skills/minimal-api-endpoints/SKILL.md` and read the lines between the two `---` markers at the top. That header is the skill's front matter, and it holds a `name` and a `description`:

```
---
name: minimal-api-endpoints
description: "Conventions for ASP.NET Core Minimal API endpoints: endpoint files, TypedResults, error responses,
---
```

At startup, `pi` puts the name and description of every installed skill into its system prompt, together with the path to the file. The body below the header stays on disk. The agent reads it only when a task matches the description. So the description costs tokens on every turn, and the body costs nothing until it is needed. That is why a skill can run to a few hundred lines while `AGENTS.md` has to stay short.

Read how the description ends: "Use when creating the API project, adding or reviewing endpoints, or writing endpoint tests." That sentence names situations, and the agent compares its task against them. A small model still skips a matching skill now and then, which is why `AGENTS.md` in the next chapter names each skill together with the task that loads it.

You can also force a skill. `pi` registers every skill as a command, so typing `/skill:minimal-api-endpoints` in a session loads that body right away, and whatever you type after the name becomes the request. You will use that later to test a skill you wrote yourself.

3.2 AGENTS.md

The skills are installed, but the agent still has to decide to use them. Until it does, `pi` sees only each skill's name and one-line description. `AGENTS.md` is the file `pi` loads into context at the start of every session. The rules that say which skill to use, and when, belong there. Keep the file short. A long `AGENTS.md` is an ignored `AGENTS.md`.

Create `~/work/AGENTS.md` and type this into it:

````
# Agent instructions

## Up-to-date documentation

Your training data is older than the tooling in this container. Before you write
.NET, C#, EF Core, ASP.NET Core or any other Microsoft-related code, load the
`microsoft-docs` skill and look the topic up. There is no MCP server in this
container, so use the skill's CLI fallback:

```sh
npx @microsoft/learn-cli search "<query>"
npx @microsoft/learn-cli fetch "<url>"
```

Never guess an API, a project file property or a CLI flag. Verify it in the docs.

## .NET Project conventions

- Target .NET 10 and C# 14.
- Solutions are `.slnx` files, never the old `.sln` format.
- HTTP endpoints are Minimal APIs, never MVC controllers.
- Extend types with C# 14 `extension(...)` blocks, never with classic
  `this`-parameter extension methods.
- Every NuGet package reference must be on its latest stable version.
- Write C# the way the `general-csharp-guidelines` skill describes.

## Layout

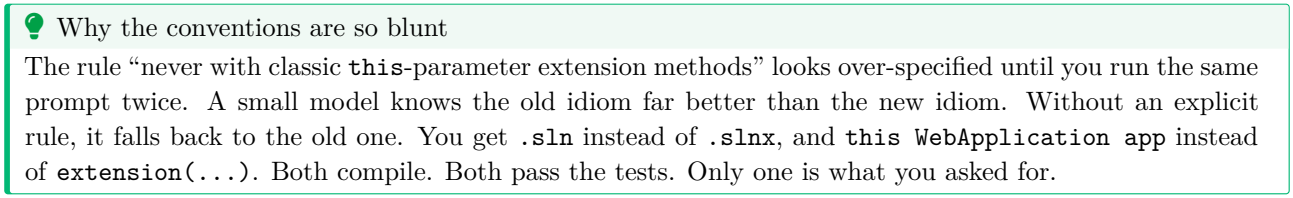
- The application is called `Calculator`. The solution is
  `backend/Calculator.slnx`. Next to it sit two projects,
  `backend/Calculator.Api` and `backend/Calculator.Tests`. Do not add a class
  library for services or data access. `.editorconfig` stays in the
  repository root.
- Before you create a project or write an endpoint, load the
  `minimal-api-endpoints` skill and follow it, with `Calculator` as `{App}`.
- Quality checks live in one script at `scripts/quality-gate.sh`.
````

3.2.1 Why this file works

Four rules make this file work.

- **It names the skills, and when to load them.** The agent now knows the documentation lookup exists instead of guessing from training data that is out of date. It also knows which skill to read before it creates a project, instead of hoping a one-line description catches its attention.
- **It names the fallback.** The Model Context Protocol server the skill prefers is not running in this container. The agent does not waste a turn finding that out.
- **It states the conventions once,** in a file that loads into every session. You never repeat them in a prompt. The agent can't forget them halfway through the course.
- **It pins the name and the paths, and leaves the rest to a skill.** Commands in later steps depend on exact paths. `dotnet build backend/Calculator.Api/Calculator.Api.csproj` and `bash scripts/quality-gate.sh` only work because the agent puts the files where the course expects them. The skill's layout writes the application name as `{App}`, and the solution that would tell the agent the real name does not exist before the first prompt. So `AGENTS.md`, the one file the agent reads on every turn,

names the application and the project paths. It also rules out the class libraries a larger backend would get, so the agent keeps the endpoints and their logic in the API project. The folders inside the projects and the naming rules stay in the skill. One of those rules stops a C# trap. Outside its own namespace, the bare name `Calculator` means the root namespace, even with a `using` for the class. A class called `Calculator` turns `Calculator.Add(2, 3)` in an endpoint into error CS0234.



3.3 The first prompt

The `microsoft-docs` skill gives the agent a way to look things up. The backend skeleton skills describe the project, and `AGENTS.md` tells the agent when to read each one. Now find out whether the skills and `AGENTS.md` are enough. Start `pi` in `~/work` and paste this prompt:

```
Set up the backend skeleton for a small web application in this folder.

- One solution with an ASP.NET Core Web API project and an xUnit test project.
- Add an `Add` operation for two numbers, and a unit test for it.
- Add an API endpoint in its own file that calls `Add` and returns the result.
- In launchSettings.json, set applicationUrl to http://0.0.0.0:5074 so the API
  listens on all interfaces. The browser runs on the host, outside the container.

Finally: git init, write a sensible .gitignore, add everything, and commit.
```

3.3.1 What the prompt leaves out

The prompt specifies no file layout and no `csproj` contents. It contains no code and no command lines. It says nothing about .NET versions or idioms. The prompt states the goal and nothing else. The agent works out the rest.

That split is the point. Prompts describe the task, and the harness describes the rules. A rule you put in a prompt applies to one turn. A rule you put in `AGENTS.md` applies to every turn for the rest of the project.

3.3.2 Watch it work

Watch the terminal for this run, because you will see the agent consult the docs instead of guessing. You should see the agent do most of this:

- run `npx @microsoft/learn-cli search ...` for `.slnx` files, then for extension blocks and Minimal APIs
- run `dotnet new` and `dotnet build`, then `dotnet test`
- check package versions against NuGet
- possibly start the API and call it with `curl`

Nobody told the agent about the `dotnet new sln --format slnx` flag. Nobody showed it what a C# 14 `extension(...)` block looks like. It read the documentation because `AGENTS.md` told it to load the `microsoft-docs` skill. That skill showed it how to query the docs.

3.4 Review the result

The agent says it is done and it has committed. Now you check, because a summary written by the agent is not evidence about the agent's own work.

3.4.1 Run the checks yourself

From `~/work`:

```
git log --oneline
git show --stat
dotnet build backend/Calculator.slnx
dotnet test backend/Calculator.slnx
dotnet list backend/Calculator.slnx package --outdated
```

The last command matters most. `AGENTS.md` says every NuGet package reference must be on its latest stable version. The agent will happily tell you it checked. `dotnet list package --outdated` is how you find out whether that is true. It prints one line per package that has a newer stable version. When everything is current, it names no packages at all.

3.4.2 Find out what the agent built

The prompt never named the solution, the class that adds two numbers, or the endpoint file. Those names are the agent's own invention, so yours will differ from your neighbor's. Read them:

```
git show --stat
ls -R backend
```

Then check four properties. Each one is something `AGENTS.md` demanded. Each one is also something a model trained before .NET 10 gets wrong when nobody stops it.

- **The projects sit where you pinned them.** Two projects side by side, `backend/Calculator.Api` and `backend/Calculator.Tests`, and no class library next to them. The `Add` logic sits in the endpoint file under `Calculator.Api/Endpoints`, as a method the unit test calls directly. This one is not negotiable, because every command later in the course is written against those paths.
- **There is exactly one solution file, `backend/Calculator.slnx`.** If you find a `.sln`, the agent used the format it knows from training. That format is not the one the harness asked for.
- **The endpoint lives in its own file,** not inside `Program.cs`. Find that file in the `git show --stat` output.
- **That endpoint file uses a C# 14 `extension(...)` block.** Open it and look at the signature. You want a block that names the extended type once, in the shape `extension(IEndpointRouteBuilder app)`. The mapping method is written inside it. A method whose own parameter list starts with `this` is the classic extension method idiom. That means the agent fell back to the old style. It compiles and the tests pass, but it is still not what you asked for.

If a property fails, don't fix it by hand. Tell the agent what is wrong and let it repair its own output. Patching it yourself hides the failure and teaches the harness nothing.

3.4.3 What this step taught you

What went right. Three skills and a handful of lines in `AGENTS.md` were enough. A small model used them to write current, idiomatic .NET 10 code. The context did the work, not the prompt.

What is still broken. The checks so far do not tell you whether the code is any good.

- `dotnet build` succeeded, which proves the code compiles and nothing else.
- The agent told you the tests pass. You are believing a report written by the thing under test.
- If you were lucky, the agent called the endpoint with `curl`. A check that depends on luck proves nothing.
- Nothing looked at formatting, naming, or dead code. Nothing checked whether the code follows a single convention, beyond the four properties you just checked by hand.

Every one of those checks currently depends on the agent deciding to run it. That decision is not deterministic. Run the same prompt again and you get a different set of checks. The next part takes the decision away from the agent entirely. Analyzers turn sloppy code into build errors. An extension runs the checks whether the agent wants them or not.

Part 4

The quality gate

4.1 Analyzers and a gate script

Right now the agent decides whether your code gets formatted and whether the analyzers run. It decides whether the tests run too. The same prompt leads to a different decision on a different day. When the agent does run the tests, you learn the result from a summary the agent writes about its own work.

A check that only runs when the agent chooses isn't a gate, because a gate runs every time. The agent might run it today, but that tells you nothing about tomorrow.

4.1.1 What a real gate has to do

1. It runs every time, without being asked.
2. It produces a machine-readable verdict, not prose.
3. It is hard to satisfy by accident. A warning has to be a failure.

The third requirement matters because a warning is easy to ignore. If a rule violation only prints a yellow line that scrolls past, the agent can honestly report "the build succeeded." Broken code still sits in the project. Turn the warnings into errors and that sentence becomes impossible to write.

4.1.2 One script first, then something that runs it

This part builds the gate in two moves. This chapter gives you a shell script that runs the checks and prints a machine-readable verdict, plus a build that treats warnings as errors. The automation that runs the script every time comes later. The script comes first because a gate that only the agent can trigger is miserable to debug, so it has to work from your shell before anything calls it.

4.1.3 Start this one in a fresh session

Commit first, so `git status` comes back clean. Then leave `pi` and start it again, or type `/new`.

The old conversation is full of `dotnet new` output and NuGet version lookups from building the skeleton. The agent would carry all of it in its context window through the longest run in the book. Discarding it is safe, because everything the agent needs is in `AGENTS.md`, the committed code, and `git log`. None of it lives in the chat.

So here is the rule for the rest of the book: new task, new session. Continue a session while you review what the agent just did or tell it to repair what it got wrong. Start fresh when the task changes.

When a fresh session produces a worse result than a continued one, that is a finding. Something the agent needed was only in the conversation, and it belongs in `AGENTS.md` instead.

4.1.4 The prompt

Paste this prompt into the fresh session. `AGENTS.md` already names `scripts/quality-gate.sh`, so your first run may have left a small script there. That changes nothing, because the prompt defines what the script has to do and the agent extends or replaces whatever is in place.

```
Add a deterministic quality gate to this project.

1. Turn on .NET code analysis for every project in the solution: recommended
   analysis mode, warnings treated as errors, and all code style rules reported
   as warnings. Look up how to configure this properly.

2. Put the checks into one single script at scripts/quality-gate.sh. That script
   executes, in this order:
     - dotnet format, verify only, must not rewrite files
     - dotnet build
     - dotnet test
   The script must write the combined output to quality-gate.log in the project
   root. The very last line of that file must be exactly `QUALITY GATE: GREEN`
```

```
    or `QUALITY GATE: RED`, with nothing after it, so other tools can read the
    verdict. The script exits 0 on green and non-zero on red.

Do not disable or downgrade any analysis rule to make the existing code green.
Fix the code instead. Commit when done.
```

4.1.5 What the prompt does not contain

The prompt carries no MSBuild properties and no `.editorconfig` rules, and you write no bash in it. It states the requirement, and the agent works out the mechanism from Microsoft's documentation through the `microsoft-docs` skill.

Read these two parts of the prompt twice.

The last line of the log file. "Exactly `QUALITY GATE: GREEN` or `QUALITY GATE: RED`, with nothing after it" is the machine-readable verdict. Everything you build in the rest of this part reads that string or the script's exit code.

The ban on downgrading rules. Turning on every style rule makes existing code fail. The fastest way to stop the failures is to switch the rule off. Write that ban into the prompt, or you'll get a green gate that checks nothing.

Expect this run to take a while, even though a large frontier model would finish it quickly. Treating warnings as errors means that code which built fine a minute ago now has dozens of errors. The agent has to fix every one of them before it can claim success. That's the gate doing its first run against the code you already have.

4.2 Own the configuration

The agent just wrote three pieces of harness for you: the MSBuild settings that turn analysis on, the `.editorconfig` that holds the style decisions, and the gate script at `scripts/quality-gate.sh`. In a few minutes you'll trust those three files to tell you whether your code is acceptable. So read them first.

Start by finding out what actually changed:

```
git show --stat
```

Open every file in that list. The sections below tell you what to look for in each one. Your files will not match another run's word for word, and they don't have to. What has to match is the effect.

4.2.1 The build has to turn warnings into errors

MSBuild automatically picks up a file named `Directory.Build.props` from the folder tree above a project. That makes it the natural place for settings that apply to the whole solution. Look for that file first. If the agent put the properties into the individual `.csproj` files instead, that works too. You just have more places to keep in sync.

There is more than one correct way to write these settings. Read your file for the four effects in the table, not for the exact property names. `AnalysisLevel=latest-recommended` and `AnalysisMode=Recommended` are the same decision spelled differently. "All style rules as warnings" is often a `dotnet_analyzer_diagnostic.category-Style.sev = warning` line in `.editorconfig`.

Effect you are looking for	One way to write it
The recommended `CAxxxx` code-quality rules are on	`AnalysisMode=Recommended`
All `IDExxxx` code-style rules are on	`AnalysisModeStyle=All`
Style rules run during `dotnet build`, not just in the IDE	`EnforceCodeStyleInBuild=true`
Any warning fails the build	`TreatWarningsAsErrors=true`

People miss the third row. Style rules are an IDE feature by default. Without `EnforceCodeStyleInBuild`, your `IDExxxx` findings never reach the build output, so a gate that runs `dotnet build` never sees them. If you cannot find that effect anywhere, search for it:

```
grep -rn "EnforceCodeStyleInBuild\|TreatWarningsAsErrors" --include=*.props --include=*.csproj .
```

If that grep prints nothing, the gate you are about to turn on is weaker than the prompt asked for. Send the agent back before you continue.

4.2.2 The editorconfig is where cheating shows up

`.editorconfig` holds the concrete style decisions: indentation, brace placement, `var` usage, and the order of `using` directives. Microsoft's C# language rules documentation describes every option in the file. The agent should have read that page while writing it.

It is also the first place to check for cheating:

```
git diff HEAD~1 -- .editorconfig backend/Directory.Build.props | grep -i "severity = none"
```

Any `severity = none` in that diff is the agent deciding which rules you get without telling you. Some of those decisions are fine. Test method names genuinely break `CA1707`, the rule about underscores in identifiers. Turning it off for the test project is a fair choice. But you make that decision, not the agent, and only after it has told you why. A rule switched off in silence is a rule you will never think about again.

> **Review the code the agent changed**
>
> Read the C# source diff as well as the configuration. The agent may have replaced file-scoped namespaces with block-scoped ones (IDE0160). Look for expression-bodied methods rewritten with block bodies (IDE0022), and `var` replaced by explicit types (IDE0008).

Decide whether those changes match your team's preferences. You can keep the style or declare a different preference in `.editorconfig`. Explain your decision before asking the agent to apply it. A green gate proves the code follows the configured rules; you still have to choose those rules.
Allowing either `var` or an explicit type requires a separate decision. `IDE0007` enforces `var`, and `IDE0008` enforces explicit types. To accept both, disable both diagnostics in the intended scope and explain why.

4.2.3 The gate script

`scripts/quality-gate.sh` is the file you will read most often, so read it now while it is small. It runs the three stages in order and writes everything to `quality-gate.log`.

Check four things in your version.

The format stage only checks. The prompt asked for verify mode, so the script should pass something like `--verify-no-changes` to `dotnet format`. A gate that fixes the formatting itself reports green and teaches the agent nothing.

The last line is the verdict, with nothing after it. No timestamp and no row of dashes. In later chapters, other programs read that verdict, and any trailing line breaks them.

The script stops at the first failing stage. You want the first problem, not every failure that follows it. A later chapter relies on the log holding only the stage that failed. If your script runs all three stages whatever happens, send the agent back.

The exit code follows the verdict. Run it yourself and look:

```
bash scripts/quality-gate.sh; echo "exit code: $?"
```

Right now, on committed code, it should print `0`. Break a file and it has to print something else. The automation you add in the next chapter decides green or red from that number alone. A script that always exits 0 gives you a gate that can never fire.

Why one script and not three commands in the harness

Nothing forces the checks into a script. Whatever runs the checks could run `dotnet format`, `dotnet build`, and `dotnet test` separately. Putting them in one file gives you a single verdict and a single log. You can also run the same gate from your shell and later from a build server. A gate that only exists inside the agent's session cannot be rerun when it fails.

4.3 Automate the gate

`scripts/quality-gate.sh` covers two of the three properties from the gate requirements. It prints a machine-readable verdict, and with warnings turned into errors it is hard to satisfy by accident. Property 1 is still missing. Right now the gate runs when you type its name, and never otherwise.

4.3.1 What a pi extension is

`pi` loads TypeScript files from `.pi/extensions/` in the project and runs them at session start. An extension subscribes to lifecycle events. When one of those events fires, your code runs inside the agent's process. It can run shell commands and read files. It can also put a message into the running session.

Other harnesses call the same idea a hook. Claude Code, OpenAI Codex, and GitHub Copilot all let you attach programs to fixed points in a session. The idea is the same everywhere, even though each API is different.

The event you want is `agent_end`. It fires when an agent run ends. That is the moment `pi` decides whether it is finished or has more to do. A run is what `pi` calls one uninterrupted stretch of agent work. A single prompt from you can turn into several runs. That timing is what matters, and it is easy to get wrong.

You won't write that file. You prompt for it and then read it, which works the same way as everything else in this course.

4.3.2 The prompt

Commit what you have and start a fresh session, then paste this:

```
Nothing runs the quality gate automatically. Add that as a project-local pi
extension, written in TypeScript.

What it has to do:

- When your run ends, the extension runs `bash scripts/quality-gate.sh` itself.
- Green gate: nothing happens. No message, no output, silence.
- Red gate: the extension feeds the failures from quality-gate.log back into the
  session as a follow-up user message, and the agent picks the work up again on
  its own and fixes them. Nobody types anything.
- The follow-up message tells the agent to fix the failures and then end its
  turn, without running the gate itself. The extension runs the gate again
  when that run ends.

Rules:

- Hook into the `agent_end` event. `agent_settled` looks like the right one and
  is not: it fires only after pi has decided it will not continue, so a message
  sent from there never starts a new turn.
- Your handler must wait for the gate to finish and send the message before it
  returns. Do not start the gate and return immediately. pi decides whether to
  continue the run from what is queued when the handler resolves, and a handler
  that finishes its work later talks to a session that has already moved on.
- Look everything else up in pi's own documentation. Never guess a function
  signature.
- Do not add any npm package. No new dependencies.
- Do not change scripts/quality-gate.sh.
- The gate takes minutes, so give it a long timeout.
- Keep the injected text small, a few dozen lines of the log at most.

Commit when you are done.
```

4.3.3 Two of those rules are not requirements

Most of the rules are outcomes, the way every prompt in this course states outcomes. The first two are different. They name an event and a control-flow rule, the kind of detail you normally pay the agent to work out.

A small model left to the outcome-only version of this prompt tends to pick `agent_settled`, or it returns from the handler before the gate finishes. Either way, a red gate then produces no follow-up at all. A larger and more expensive model given the same prompt experiments on its own harness. It picks an event and breaks the gate on purpose to see whether the run continues. When the run stops, it reads the transcript and finds its own mistake.

So every prompt holds two kinds of sentences. Some are the requirement and stay true whatever model you run. Others compensate for the model you happen to use, and you delete them when you move to a stronger one. The two technical rules are the second kind. This course pays that price on purpose, in exchange for a model that fails where you can see it.

4.3.4 The prompt names no documentation

The analyzer prompt sent the agent to Microsoft's documentation. This one only says "look everything else up in pi's own documentation", with no path and no file name. That is enough, because `pi`'s system prompt already tells the agent where its own documentation lives and which file covers which topic.

Before you paste a documentation path into a prompt, ask the tool where its manual is. It may already know.

4.3.5 Read what the agent wrote

```
git show --stat
ls .pi/extensions/
```

You should see one TypeScript file. Its name is the agent's choice. That is why the greps in this section search the whole folder.

The event has to be `agent_end`. This is the one check you can do with grep. It also decides whether the loop works at all:

```
grep -rn "pi.on(" .pi/extensions/
```

One of those calls has to register `agent_end`, and none of them may register `agent_settled`. A second handler on `session_start` that resets a counter is fine. So is the word `agent_settled` in a comment. You will probably find one there. The prompt named the trap, and the agent tends to write down why it avoided it. `agent_settled` inside `pi.on(...)` is the one that matters. Your gate runs and goes red, and no follow-up message appears.

The handler has to be async and finish its work before it returns. Open the file and find the line that registers the handler. A working one reads:

```
pi.on("agent_end", async (_event, ctx) => {
```

Then follow the body. The call that runs `scripts/quality-gate.sh` has to be awaited. The call that sends the message back into the session has to come before the handler returns, not after it in some callback. If the body starts the gate and returns right away, you have the second trap. `pi` will crash the first time the gate goes red.

The injected text has to be bounded.

```
grep -rniE "slice|tail|lines|truncate" .pi/extensions/
```

A working version keeps the last 40 lines of `quality-gate.log` and throws the rest away. That matters more than it looks. Whatever the extension injects lands in the agent's context window on the next turn. A build log with a thousand error lines spends the budget the agent needs for the actual fix. If yours pastes the whole file, send it back and ask for a cap.

The message has to tell the agent to stop after the fix. Open the file and read the text the extension sends. It has to say that the agent ends its turn once the failures are fixed, and that the agent does not run the gate itself. "Continue working until the quality gate passes" sounds harmless, but it invites the agent to run the gate as its last action to confirm the fix. Then the run ends, `agent_end` fires, and the extension runs the whole gate a second time. The extension's run is the one that counts, so the agent's run only costs you minutes.

4.3.6 Extensions load at session start

The file exists and the extension still does not run. `pi` reads `.pi/extensions/` once, at session start. The session that wrote the file started before the file was there.

Leave `pi` and start it again, or run `/reload`. Do that after every change to the extension, including the ones the agent makes for you.

Two more things stop an extension from loading. The project has to be trusted. That is the trust prompt `pi` shows on the first start in a new project. Accept it, or start with `pi -a`. Project-local extensions only come from `.pi/extensions/`. An extension the agent decided to put somewhere tidier is an extension that never runs.

You don't just have to believe this. The next chapter proves the extension is loaded by watching it do its job.

> **i** When the session compacts itself
>
> Long runs fill up the context window. `pi` then summarizes the older half of the conversation on its own. You see it happen in the transcript. Let it finish, and do not abort in the middle of a fix.
> `/compact` triggers the same summary by hand, and you rarely need it. When a run has gone on so long that you want to use `/compact`, commit whatever is already good. Start the next attempt in a fresh session instead.
> The signal to restart is the agent repeating itself, or drifting away from a rule you put in the prompt. A summary keeps the shape of a conversation, but it loses the details. "Do not disable any rule to make the code green" is exactly the kind of detail that goes missing.

4.4 Close the loop

The extension is written and committed. You loaded it into a fresh session. Everything you know about it so far comes from reading it. Reading does not tell you whether it works.

4.4.1 An untested gate is worse than no gate

> **!** Keep this rule in mind
>
> An untested gate is worse than no gate. It gives you confidence you have not earned. A gate you have never watched fail is a green light with no bulb in it.

The rule applies to the extension's two behaviors, and you have to see both. It runs the gate and stays quiet when the code is fine. It puts the agent back to work when the code is not. Test green first, because it is the cheaper experiment.

4.4.2 Prove the green path first

Commit, so the working tree is clean and the gate is green. Then start `pi` and give it a prompt with no work in it at all:

```
Reply with the single word ok. Do not use any tools.
```

The agent answers `ok`, and that is the entire session output. You see no gate banner and no verdict line, and the agent never gets a second turn. From the transcript alone you cannot tell whether anything ran.

It is running, though. The gate takes a minute or two after `ok` comes back, and none of that shows up in the session. Give it that long, then leave `pi` and look at the log instead:

```
ls -l quality-gate.log
tail -1 quality-gate.log
```

The timestamp on the file moved to a few seconds ago, and the last line reads `QUALITY GATE: GREEN`. The format stage ran, then the build, then the tests. All of that happened on a prompt that did nothing. The harness stayed quiet about it.

Silence is the green state, and that is a design decision you should copy. A message the agent reads on every turn, and can never act on, is noise in the context window. A gate that announces good news is a gate people stop reading.

4.4.3 Break the code by hand

Now make the gate fail. Open the class that adds two numbers. You found its file when you reviewed the first commit in the context part, so go to the same file now.

Indent the `return` line by four extra spaces and delete the spaces around the `+` operator. Save it, and leave the working tree dirty.

You break the code by hand rather than asking the agent to write sloppy code on purpose. Models are trained to produce clean code, so they tidy up even when you tell them not to. Ask for broken code and you spend the experiment arguing about whether the code was actually broken.

4.4.4 Read the evidence

Before you let the harness see it, run the gate yourself and read what it says:

```
bash scripts/quality-gate.sh
tail -30 quality-gate.log
```

The gate stops at the first stage that fails, the property you checked when you read the script. A formatting break never gets as far as the analyzer or the tests, so the format stage is the one complaining.

Your file names differ from everyone else's, and so do your line numbers and rule ids. Two properties of the output are the same for everyone.

Every finding carries a file, a line and column, plus the rule that fired. That is machine-readable evidence rather than an opinion. You can grep it and count it. You can also hand it to another program, which is exactly what the extension is about to do next.

Nothing in this log is advisory. A warning is something an agent can report and then ignore. Every line in this log belongs to a stage that failed. That stage failed because the tool it ran returned a non-zero exit code. In the build stage, `TreatWarningsAsErrors` does the work: a style violation becomes an error, and `dotnet build` fails. The script stops there.

The last line of the file is the verdict, `QUALITY GATE: RED`, with nothing after it.

4.4.5 Watch the loop close

Start `pi` again and send the same empty prompt as before:

```
Reply with the single word ok. Do not use any tools.
```

Then take your hands off the keyboard. This one takes several minutes, because the gate runs twice. Watch this sequence happen, with no human input:

1. The agent replies `ok` and its run ends.
2. The extension runs the gate. The verdict comes back RED.
3. The tail of `quality-gate.log` goes into the session as a follow-up message.
4. The agent starts working again, finds the over-indented line and the missing spaces around the `+`, and fixes the file.
5. That run ends, and the extension runs the gate again. This time it comes back GREEN.
6. The session goes quiet, with a clean working tree.

The agent's last message claims the fix in its own words. Leave `pi` and check that it is telling the truth:

```
tail -1 quality-gate.log
git status --short
```

Stop and appreciate what just happened. Nobody asked for that repair. You did not read the log, and you did not paste it into a prompt. The agent never decided that this change was worth checking. A file in `.pi/extensions/` ran a script. The script compared a number to zero, then put the failure back in front of the agent.

Suppose nothing happens after `ok`, and the log still holds the RED verdict you produced by hand. Then the extension never loaded, or it listens on the wrong event. Restart `pi` and re-run the grep commands from the "Automate the gate" chapter. A fresh timestamp on the log with no follow-up points to the wrong event. An old timestamp means nothing ran at all.

Suppose instead the log says GREEN while the file is still broken. Then the script is setting the verdict to GREEN before it captures the build output. Re-run `bash scripts/quality-gate.sh` by hand and read what it wrote.

4.4.6 What the loop costs

The agent does not stop when you think it stopped. A red gate spends another turn and a few more minutes of gate runtime, without you asking for either. On a clean run you never notice. On a bad one you come back to a session that has been arguing with itself.

The repair and the original change land in the same session. You want to read a diff at the point where the agent finished. But by the time you look, two more commits may already sit on top of it. Read `git log` before you read the code.

A gate that can never go green never stops. The agent sees RED, and the extension sends a follow-up. The agent tries again, but the gate stays RED, so the cycle repeats. The extension needs a hard limit on consecutive retries, a retry ceiling.

4.4.7 The retry ceiling

The loop assumes the agent can fix what the gate reports. Sometimes it cannot. A stopped PostgreSQL container makes the gate go red for a reason that has nothing to do with any line of code. No C# edit will bring it back to green. An extension that answers every red verdict with a follow-up message would sit there retrying until you kill it.

So find out whether your extension has a ceiling:

```
grep -rniE "max|consecutive|attempt|count" .pi/extensions/
```

A working extension stops after three consecutive red verdicts. It resets the counter when a session starts, and it shows a warning once when it trips. The prompt did not ask for a ceiling, so your agent's version might not have one.

If your grep comes back empty, prompt for a ceiling:

```
The quality gate extension keeps sending follow-up messages as long as the gate
is red, which never ends when the failure is not something the agent can fix.
Add a ceiling: after three consecutive red verdicts, stop sending follow-up
messages and show a warning instead. Reset the counter when a session starts and
whenever the gate comes back green.
```

Restart `pi` afterwards, because extensions load at session start.

Three wasted turns and a warning make a bad afternoon. An unbounded repair loop against a stopped database makes a bad week.

4.5 Tell the agent

The agent still does not know that `scripts/quality-gate.sh` exists. So it invents its own verification, decided fresh every turn: a `dotnet build` here and a `dotnet test` there, or nothing at all. Give it the script you own as the entry point instead.

The note also has to say when not to run the gate. If the agent runs it as its last action, the extension runs the whole gate again right after, and every check runs twice.

4.5.1 Add the section to AGENTS.md

Type this into `AGENTS.md` yourself, at the end of the file:

```
## Quality gate

`bash scripts/quality-gate.sh` runs format, analysis, and tests. The verdict is
the last line of `quality-gate.log`.

The gate also runs by itself when your run ends, from the project-local pi
extension in `.pi/extensions/`. A red verdict comes back to you as a follow-up
message, and you fix it before you are done.

- Run it after an intermediate step of a longer task, to catch problems early.
- Do not run it as your final action. It runs automatically when your run
  ends, and it is slow.
```

> Tell the agent about every automation you add
>
> Each stage you add to the gate is one more check the agent might run by hand, right before the extension runs it again. Whenever you automate something, say in `AGENTS.md` that it exists and is already handled.

4.5.2 What this step taught you

The gate now runs after every agent turn, whether the agent wants it to or not. The verdict is a string in a file and an exit code rather than a paragraph of self-assessment. A red verdict comes back into the session and puts the agent back to work. That is a deterministic gate with a closed loop. You built it from one script and one extension, and you never wrote a line of either.

The gate you just built has four gaps.

Nothing prevents a red commit. The gate checks only after the commit already happened. It cannot block anything before that point. When the agent commits and the gate goes red, that commit is already in your history. The loop then fixes the code with a second commit instead of stopping the first one.

The gate is slow. It runs after every turn, even for a one-line change. A full run already takes minutes. EF Core and Angular make the build slower, and Playwright makes the test run longer.

The gate only covers the backend. It skips linting and frontend tests. It has no integration tests and no database.

`TreatWarningsAsErrors` with every style rule enabled is brutal. It turns every style opinion into a build failure. The rest of the course has to find where a gate that strict costs more than it saves.

Those gaps are the agenda for the rest of the course.

4.5.3 Where you stand

Harness element	After the context part	After this part
Context	`AGENTS.md` plus three skills	unchanged
Tools	`mslearn` CLI, `dotnet`, `git`	unchanged
Gates	none, the agent decided	format, analysis, and tests, always

Harness element	After the context part	After this part
Feedback loop	you, reading a summary	automatic, the agent fixes red before it stops

All four elements now exist, but none of them is finished. The next part gives the backend a real database. It also gives the gate a fourth stage, one that catches a kind of problem no compiler and no test will ever report. That problem is a generated file that no longer matches the code it was generated from.

Part 5

The environment

5.1 Persistence

5.1.1 When the gate grows a dependency

Every stage of the quality gate so far ran on nothing but the source tree: a formatter, a compiler, and unit tests. Hand somebody the repository and the .NET SDK. `scripts/quality-gate.sh` gives them exactly the verdict it gives you.

A database ends that. The backend now needs a running server before it can serve a single request, and the integration tests run against that server, so the gate needs it running too. From here on, your quality gate can go red for a reason that has nothing to do with your code.

You add this dependency on purpose. Do it now, while the project is still small. At this size, you can trace exactly what the gate needs from the environment.

5.1.2 Split the backend into layers

The calculator started as one API project because it had no data to look after. With a database it has rows to store and queries that belong together, and keeping those out of the endpoints is the job of a service layer. The backend grows to four projects. `Calculator.Api` keeps the endpoints. `Calculator.Services` gets the business logic, and `Calculator.Persistence` gets the DbContext, the entities, and the migrations. `Calculator.Tests` stays where it is.

The rules for that layout come in two more custom skills, from the `persistence` folder in `~/skill-source`. Install them the same way you installed the first two:

```
cd ~/work
npx skills add ~/skill-source/persistence -y
ls .agents/skills/
```

Two new folders appear. `service-layer` says which project owns what, and how an endpoint calls a service. `ef-core-postgres` covers the EF Core side, down to the flags every `dotnet ef` command needs once the DbContext lives in its own project.

Installing them is only half the change. `AGENTS.md` still tells the agent that the backend has two projects and no class library. `pi` loads `AGENTS.md` on every turn, while a skill loads only when the agent picks it, so the old rule would win. Open `~/work/AGENTS.md` and replace its `## Layout` section with this one:

```
## Layout

- The application is called `Calculator`. The solution is
  `backend/Calculator.slnx`. Next to it sit four projects:
  `backend/Calculator.Api`, `backend/Calculator.Services`,
  `backend/Calculator.Persistence`, and `backend/Calculator.Tests`.
  `.editorconfig` stays in the repository root.
- Before you create a project or write an endpoint, load the
  `minimal-api-endpoints` skill and follow it, with `Calculator` as `{App}`.
- Before you add a service, an entity, or a migration, or run any `dotnet ef`
  command, load the `service-layer` and `ef-core-postgres` skills and follow
  them, with `Calculator` as `{App}`.
- Quality checks live in one script at `scripts/quality-gate.sh`.
```

The sentence that forbade a class library is gone, and the layout names four projects, because later commands in this part depend on those paths. The new third bullet tells the agent when to load the two new skills. Their one-line descriptions alone are easy for a small model to skip.

Commit the new context before you prompt, so the persistence commit holds only what the agent did:

```
git add AGENTS.md .agents skills-lock.json
git commit -m "Add the persistence skills and the four-project layout"
```

5.1.3 The prompt

The PostgreSQL server is already running. `compose.yaml` starts it as the `db` service. The container you work in gets the connection details as environment variables. Check them before you prompt:

```
printenv | grep '^PG'
```

Then give `pi` this prompt:

```
Add persistence to the backend.

- Use EF Core with the PostgreSQL server that already runs in this environment.
  Its connection details are in the PGHOST, PGPORT, PGDATABASE, PGUSER and
  PGPASSWORD environment variables of this container.
- Add a `Values` entity with an `Id` and two int properties named `Left` and
  `Right`. The JSON the API accepts and returns uses `left` and `right`.
- Use EF Core migrations and apply pending migrations automatically on backend
  startup.
- Add a new API module in its own file for `Values`, with: create, list all
  (ids only), get by id, and delete.
  - Get by id must also return the sum of the two values, computed with the
    existing `Add` logic.
- Then extend scripts/quality-gate.sh with a stage that fails when the entity
  model has changed but no migration covers that change. Use the command that
  EF Core ships for exactly this check,
  `dotnet ef migrations has-pending-model-changes`. It needs no database
  connection, so put the stage where it runs even when the database is down.

Look everything up in the docs. Commit when the quality gate is green.
```

5.1.4 What the prompt pins down

The prompt states outright what no documentation could tell the agent. It names the five `PG` environment variables, because only this container hands out its connection details that way. It pins the entity name and its two properties, down to the lowercase JSON names. And it makes `get by id` reuse the existing `Add` logic instead of adding two numbers a second time.

`dotnet ef migrations has-pending-model-changes` is pinned for a different reason. A small model asked to "check for missing migrations" tends to write a homemade check instead, a grep over the migrations folder or a trial run of `dotnet ef migrations add`, and you end up maintaining it. The command ships with EF Core, and naming it costs one line.

Everything else stays open. The `ef-core-postgres` skill covers the NuGet package and the migration commands, and the agent looks up the rest. Where the code goes is settled by `AGENTS.md` and the `service-layer` skill. The DbContext, the entity, and the migrations land in `backend/Calculator.Persistence`. The `Values` service goes to `backend/Calculator.Services`, and the endpoints stay in `backend/Calculator.Api`.

That split is the point. You tell the agent what exists in this environment and what the result has to be. The documentation covers how to get there, and the gate checks the result afterwards.

> **i** The database keeps what you put in it
>
> `compose.yaml` mounts a named volume for PostgreSQL, so rows survive a restart of the container. Your database is not empty after the first run. Later chapters expect the rows you add now to still be there.

This is a long turn for a small model. Expect the model to work for a while. Expect the gate to run, and sometimes fail, before the agent gets to a commit.

5.1.5 A migration is a generated file

Every file the gate has looked at so far was typed by a person or by the agent. Migrations are not. EF Core reads your entity classes and writes the migration for you. That puts a new kind of file in the repository, and a new kind of failure with it.

A missing migration fails quietly. Add a property to the entity, and forget the migration. The project still compiles, because C# does not know about your database. The unit tests still pass, because none of them looks

at the schema. The application starts, sends its first query, and falls over at runtime because the table has no such column. Everything in the repository was green up to that moment.

`dotnet ef migrations has-pending-model-changes` asks one question: does the current entity model still match the last migration? It answers from the model snapshot that EF Core keeps in the migrations folder. That means it needs no database and no running app. It is fast enough to run after every turn.

That stage does not ask whether the migration is correct; it asks whether the migration is current. Current is the only thing a gate can check about a generated file. A migration that matches the model is wrong only when the model is wrong, and the earlier stages already gate the model.

Every generated file in a project deserves a check of this shape. Lock files, migrations, snapshot tests, and generated code of any kind all share the same weakness. A tool produced them from a source, and nothing notices when the two part ways.

5.1.6 Make the new stage fail

Never trust a gate you have not seen fail. The agent has committed and the gate is green, so break the new stage on purpose.

The agent chose the file name and the folder for the entity, so find it first. Open it, and add a third `int` property next to the other two. Add no migration. Then run the gate by hand:

```
bash scripts/quality-gate.sh
tail -20 quality-gate.log
```

The last line has to read `QUALITY GATE: RED`. Read which stage the failing header names. That is the part you are proving, and your agent wrote the wording of that header itself.

If the analyzer stage goes red first, that is warnings-as-errors reacting to a public property with no documentation comment. Give your new property the same kind of comment its neighbors have and run the gate again. Now the migrations stage is the one that fails.

There are two ways back to green. Running `git checkout` on the entity file throws the model change away. Adding the missing migration keeps it instead. Take the first one here, since the property was only ever a test. In a real run the agent takes the second one, because your gate just told it exactly what was missing.

```
git checkout backend/Calculator.Persistence  # adjust to your entity's path
bash scripts/quality-gate.sh                 # GREEN again
```

5.2 The harness pays off

5.2.1 Read the commit before you read the code

The agent has committed the persistence work. Before you open a single source file, look at what went into that commit:

```
git show --stat HEAD
```

Sort that list into two piles. One pile is what the prompt and `AGENTS.md` asked for: the two new projects with the entity, the database context, and the migration files EF Core generated, the `Values` service, and the new API module. The other pile is everything the prompt does not explain. Read that second pile first. Every file in it is either the harness at work or the agent acting on its own. You want to know which, before you trust the commit.

This commit may have such a file, and it is `.editorconfig`. Generated code collides with strict style rules. The migration generator writes a byte order mark at the top of each file it produces, and `dotnet format --verify-no-changes` rejects it. So the gate goes red on code the agent generated but never typed. The extension feeds that failure back into the session. The agent has to deal with it before it can stop. Nobody typed a word about formatting.

That is the harness paying off. Read what the agent decided to do about it:

```
git diff HEAD~1 -- .editorconfig
```

The fix you want to see is a scoped section that relaxes the rule for the migrations folder and nothing else. Exempting generated code is the one defensible version of "switch the rule off." The rule from the "Own the configuration" chapter still holds: the agent does not make that call in silence. Check the scope, not just the intent.

Not every run ends up in this exact situation, so your commit may leave `.editorconfig` untouched. The `ef-core-postgres` skill tells the agent to run `dotnet format` right after it generates a migration, and that run may already repair the file before the gate sees it. The two piles still apply. Whatever sits in your second pile, chase it down the same way: the diff, the gate log, or a direct question to the agent. A change nobody asked for is not automatically wrong, but it never gets to stay unexplained.

5.2.2 What to check in the diff

Now read the code. The agent chose the file names and the folder layout on its own, and it named the classes. Start by finding out what it made:

```
git show --stat HEAD
grep -rn --include=*.cs "MigrateAsync\|Migrate()" backend/Calculator.Api
grep -rn --include=*.cs "PGHOST\|Npgsql\|UseNpgsql" backend
grep -rn --include=*.cs "DbContext" backend/Calculator.Api/Endpoints
```

Check four properties, and each one tells you where to look.

Endpoints call services. The last grep has to print nothing. An endpoint in `Calculator.Api` takes a service interface from `Calculator.Services` and never the DbContext itself. A hit means the agent skipped the service layer for that endpoint, so send it back to the `service-layer` skill.

Migrations run on startup. `Program.cs` has to apply pending migrations before the app starts serving requests. It usually does that with a call to `MigrateAsync()` on the context's `Database` facade. Without it, the first request hits tables that do not exist yet.

The connection comes from the environment. The five `PG` variables hold the connection details. If you find a full connection string sitting in `appsettings.json`, the agent hard-coded the credentials instead. The prompt told it to read them from the environment. That setup breaks on any machine with a different database.

Get by id reuses the existing logic. The sum in the response has to come from the `Add` method, which already has unit tests. With the service layer in place, `Add` moves out of the endpoint file into a service in `Calculator.Services`. If you find `left + right` written out a second time inside the new service or endpoint, the agent duplicated that logic. Say so and make it redo that part.

5.2.3 Try it for real

Reading the diff is not the same as seeing the API work. Start the API in the container shell:

```
dotnet run --project backend/Calculator.Api/Calculator.Api.csproj
```

Then, in a second terminal inside the container:

```
PORT=<the port it printed>
# create one, and keep the id it gives back
ID=$(curl -s -X POST localhost:$PORT/api/values \
      -H 'Content-Type: application/json' \
      -d '{"left":17,"right":25}' | grep -o '"id":[0-9]*' | cut -d: -f2)
curl -s localhost:$PORT/api/values          # a list of ids, yours among them
curl -s localhost:$PORT/api/values/$ID      # {"id":…,"left":17,"right":25,"sum":42}
curl -s -o /dev/null -w '%{http_code}\n' -X DELETE localhost:$PORT/api/values/$ID
```

That block captures the id from the POST response instead of assuming `1`. The database is shared and keeps whatever earlier runs left in it. So the row you just created is almost never the first one.

The field names are the second detail to watch, and they can trap you. If your agent named the JSON fields something other than `left` and `right`, the POST does not fail. ASP.NET Core binds the unknown fields to nothing. It cheerfully stores a row of two zeros and returns 200. So check the response body, not the status code. When `{"id":…,"left":0,"right":0,"sum":0}` comes back, open the API module the agent wrote for `Values`. Read the names it actually chose. That file also carries the real route if the agent did not pick `/api/values`:

```
grep -rn "MapPost\|MapGet\|MapDelete" backend/Calculator.Api --include=*.cs
```

5.3 Integration tests

5.3.1 Install the integration test skill

The rules for integration tests come in one more custom skill, from the `integration-tests` folder in `~/skill-source`:

```
cd ~/work
npx skills add ~/skill-source/integration-tests -y
ls .agents/skills/
```

`web-integration-tests` appears next to the earlier skills. It covers the packages, the fixture that starts the API, and how a test class cleans up after itself. It also shows how the fixture moves the tests to a database of their own, which the prompt below asks for.

Tell the agent when to load it. Add this bullet to the `## Layout` section of `~/work/AGENTS.md`, below the one for the persistence skills:

```
- Before you write or change an integration test, load the
  `web-integration-tests` skill and follow it, with `Calculator` as `{App}`.
```

Commit the skill and the new bullet before you prompt:

```
git add AGENTS.md .agents skills-lock.json
git commit -m "Add the integration test skill"
```

5.3.2 The prompt

The backend now stores rows in PostgreSQL, and the unit tests still only cover the calculation logic. Nothing yet proves that a request travels through routing, model binding, EF Core, and the database, then comes back correct. That is what this prompt is for:

```
Add web API integration tests.

- Use the built-in ASP.NET Core support for integration testing. Look up what
  .NET 10 offers for this.
- Add an xUnit fixture that starts the API in-process. Make sure it applies the
  EF Core migrations before the tests run.
- The tests run against their own database on the PostgreSQL server of this
  environment, never against the development database in PGDATABASE. Name it
  after PGDATABASE with `_test` appended. The tests must not assume it is
  empty, and they must clean up whatever they create.
- Add one integration test. It creates a `Values` entity, lists all ids, gets
  it by id and checks the sum, and finally deletes it.

Commit when the quality gate is green.
```

5.3.3 The bullet that does the work

The third bullet is the one that changes how the tests have to be written.

First, it keeps the tests out of the development database. The gate runs the tests after every agent turn, and every run creates and deletes rows. In the development database those rows would sit next to the ones you created with `curl`, and one careless cleanup filter would delete yours too. A second database on the same server costs nothing: no new container and no new credentials, only a different name. With `PGDATABASE=appdb` the tests use `appdb_test`. The `web-integration-tests` skill shows the fixture how to swap the database, so the prompt only has to state the rule.

The test database can still hold rows, though. It outlives a single run, and every test class in a run shares it. Take a test that expects `GET /api/values` to return exactly one id. It passes when you run it alone. It fails the moment a second test, or a run that crashed halfway, has left a row behind. The test is not wrong about the code; it is wrong to assume the database is empty.

Writing those constraints into the prompt costs one bullet. Finding the problem later, in a test that fails only in a full run, costs an afternoon.

5.3.4 Find the pieces in your own code

The agent picked the file names and the class names again, so go look. Four things should be there, and each one is worth understanding rather than just confirming:

```
git show --stat HEAD
grep -rn "WebApplicationFactory" backend/Calculator.Tests
grep -rn "_test" backend/Calculator.Tests --include=*.cs
grep -rn "partial class Program" backend/Calculator.Api
head -1 backend/Calculator.Tests/*.csproj
```

A fixture built on `WebApplicationFactory<Program>`. This is the built-in ASP.NET Core support for integration testing. It boots the whole application in-process and hands the test an `HttpClient` wired straight into it. You never pick a port or start `dotnet run`, and the test reaches the app without `curl`. Open the fixture and confirm it also applies the migrations before any test runs.

A database of its own. The fixture builds a connection string from the same `PG` variables as the app, with `_test` appended to the database name, which is what the `_test` grep finds. It then replaces the app's DbContext registration with one that uses that connection string. The first test run creates the database, because applying the migrations creates a missing database first. List the databases on the server to see it next to the development one:

```
psql -d postgres -c '\l'
```

If only the development database is there after a green run, the tests ran against it. Send the agent back to the `web-integration-tests` skill.

The test project can name the entry-point type. Top-level statements compile into an internal class called `Program`, and `WebApplicationFactory<Program>` references it from another project. Your agent made that legal in one of two ways. The common one is a `public partial class Program {}` marker at the end of `Program.cs`, which is what the grep finds. The other is an `<InternalsVisibleTo Include="Calculator.Tests" />` entry in the API project file. If the grep comes back empty, check the `.csproj` before you send the agent back. A green build means one of the two is in place.

The factory depends on one package. The `.csproj` under `backend/Calculator.Tests` references `Microsoft.AspNetCore.Mvc.Testing`. Besides the `WebApplicationFactory` class, its build targets copy the API project's dependency manifest into the test output, and that manifest is what lets the factory boot the real app. Whether the `Sdk` attribute reads `Microsoft.NET.Sdk.Web` or plain `Microsoft.NET.Sdk` makes no difference once the package is referenced, so leave that choice to the agent.

5.3.5 Read the count, not the wording

Run the gate by hand and look at the test stage in `quality-gate.log`. The summary looks roughly like this:

```
Passed!  - Failed: 0, Passed: 10, Skipped: 0, Total: 10
```

Your wording will probably differ. If the agent picked the newer Microsoft.Testing.Platform runner, the same result reads `Test run summary: Passed!` with the counts on separate lines. The exact number depends on how many unit tests you already had. What matters is that the count went up and that the failure count is 0.

Unit tests and integration tests both run in the same stage, every time an agent run ends. You never opened `scripts/quality-gate.sh` for this step. The stage calls `dotnet test`, and `dotnet test` found the new tests on its own.

A gate defined in terms of the toolchain grows with the project. A gate defined as a list of test names does not.

5.4 Break the environment

5.4.1 Break the environment, not the code

Everything is green and the working tree is clean. Now stop the database and run the quality gate again:

```
docker compose stop db      # from your host, not inside the container
bash scripts/quality-gate.sh
tail -20 quality-gate.log
```

The tail of the log looks about like this:

```
=== Stage: dotnet test ===
   Npgsql.NpgsqlException : Resource temporarily unavailable
     at Npgsql.Internal.NpgsqlConnector.ConnectAsync(...)
RESULT: FAIL (exit code 1)

QUALITY GATE: RED
```

The exception text depends on your machine and on how far the connection attempt got. The two stable parts are the `Npgsql` name in the error and the last line of the file, `QUALITY GATE: RED`.

Nothing is wrong with your code. Not one line of it changed since the last green run. The gate is red because a container is down.

```
docker compose start db
bash scripts/quality-gate.sh   # GREEN again
```

> **Where to type docker compose**
>
> `docker compose stop db` runs on your host, in the folder that holds `compose.yaml`. Inside the development container there is no Docker daemon to talk to, so the command fails there. Everything else in this chapter runs in the container shell as usual.

5.4.2 What red means now

Until this part, a red gate meant the code is wrong. The migrations drift check stretched that a little, since a missing migration is a gap rather than a broken line. Stopping the database stretches it much further. From here on, red means the code is wrong, or something the code depends on is not there. Those two need different reactions. When the code is wrong, read the diff. When a dependency is missing, start the database. A gate that cannot tell you which one you are looking at teaches people to shrug at red. A gate people shrug at is already dead.

Your extension's retry ceiling exists for this failure. An agent cannot start a stopped container by editing code. Without the ceiling, it would keep trying until you stopped it.

5.4.3 Three things to argue about

Take these to class and disagree out loud about them. None of them has one right answer.

A gate that is slow, flaky, or dependent on infrastructure gets disabled. Not by anybody's decision, but by attrition. Somebody appends `|| true` to the failing stage on a bad afternoon. Somebody else stops reading `quality-gate.log`. Six weeks later, the gate is decoration. So what is the honest maintenance cost of the gate you just built, in seconds per turn and in patience per week?

Shared test data breaks when tests run in parallel. Your integration tests pass today because there is one of them and it cleans up after itself. Now imagine ten tests sharing one database, each creating and deleting rows. They collide with each other. What would you change first, and what does that change cost?

The agent's context is a budget. This step was split into two prompts on purpose. Give a small model EF Core, migrations, four endpoints, and an integration test fixture in a single turn. It tends to do the first half well and the second half badly. The instructions that came first crowd out the ones that came last. Splitting

work into coherent chunks is not a workaround for a weak model; it is part of designing the harness. You will do it again for the frontend.

5.4.4 The quiet win

Look at what happened while you were busy with EF Core. You added a database, four endpoints, and an integration test. At the same time, you also added the migrations drift check. It cost one bullet in one prompt, on the day the project had exactly one entity and one migration.

From now on it runs after every turn, on a project that grows entities. Every future change to the model has to arrive with a migration next to it. You have already watched the stage go red when one did not. The cheapest time to build a gate is before you need it.

Next you point the harness at a second toolchain. An Angular frontend brings its own package manager, build, tests, and documentation. The agent has seen none of it. The gate has to grow to cover all of it without becoming too slow to run every turn.

Part 6

A second toolchain

6.1 More sources of truth

6.1.1 What a frontend breaks

Everything in your harness so far runs on .NET. NuGet manages packages, and `dotnet` runs the build and the tests. `scripts/quality-gate.sh` calls `dotnet` and nothing else, and the extension runs that script after every agent turn.

An Angular frontend breaks that setup in three ways.

A second ecosystem. npm brings its own build and its own test runner. Your gate script now has to drive two toolchains and still return a single exit code.

A contract nobody generates. The Angular code that calls the backend is hand-written TypeScript. You describe the request and the response a second time, in a second language. No tool compares your description with the C# on the other side. TypeScript accepts both, so a renamed field turns into a surprise in the browser at runtime, not a compile error.

A new kind of test. End-to-end tests need the backend and the frontend running, plus a browser. They are too slow for every agent turn, so you decide which checks belong in which loop.

There's a fourth problem too. This one is about the model, not your project. `pi` runs a small model whose training data ends well before the Angular version that `ng new` installs today. It knows `NgModule`, and it always uses an observable. It has barely seen the resource API that this course uses for reads. Ask it for a service, and you'll get 2022 Angular. It compiles fine, but it's not what you asked for.

So the `angular-developer` and `find-docs` skills are not a convenience here. For the backend, `microsoft-docs` covered a difference of a version or two. For Angular, those two skills are the only place the current API exists at all. Every piece of frontend work in this part depends on the agent actually reading them.

6.1.2 Four places to find information

Your agent has had exactly one place to look up documentation: the `microsoft-docs` skill. For Angular work it needs four more. The three commands below install them.

```
cd ~/work
npx skills add https://github.com/angular/skills
npx skills add upstash/context7@find-docs
npx skills add mattpocock/skills@writing-for-agents
```

Each command stops and asks whether to install for the project or globally. Choose Project for all three, the same scope you chose for `microsoft-docs`.

Two of the three arguments use the `owner/repo@skill` form, and one is a URL. The `owner/repo@skill` form picks one named skill out of a repository that holds several, so `upstash/context7@find-docs` installs `find-docs` and nothing else. The Angular URL has no skill name, so it installs everything that repository offers. Right now, that's two skills. List the folder to check:

```
ls .agents/skills/
# angular-developer  angular-new-app  ef-core-postgres  find-docs
# general-csharp-guidelines  microsoft-docs  minimal-api-endpoints
# service-layer  web-integration-tests  writing-for-agents
```

Ten folders, four of them new. Here is what each new skill does.

Skill	What it is for
`angular-new-app`	How to create an Angular app with the CLI
`angular-developer`	Angular architecture: signals, forms, dependency injection, testing, styling
`find-docs`	Context7, current docs for any library, through `npx ctx7@latest`
`writing-for-agents`	How to write documents that an agent reads

6.1.3 Try find-docs by hand

Never install a skill you have not seen work. Run the tool that the `find-docs` skill wraps, yourself, before the agent ever calls it:

```
npx ctx7@latest library Angular
```

No API key and no MCP server are involved. What comes back is not documentation but a list of candidate library ids, each with a benchmark score. The top one for Angular is `/websites/angular_dev`, the framework's own documentation site.

Resolving a name to an id is step one of two. Step two takes the id and a question:

```
npx ctx7@latest docs /websites/angular_dev "zoneless change detection"
```

Read what that returns. It's exactly what appears in the agent's context window when it uses `find-docs`. If the output looks incomplete or wrong for a topic, don't trust the agent's answer on that topic.

6.2 Route the docs

6.2.1 Four overlapping sources need a router

Four documentation sources are worse than one unless the agent knows how to choose between them. Ask about `IEndpointRouteBuilder`, and Context7 answers right away, with docs that may or may not match your .NET version. Ask Microsoft Learn about Angular signals and you get nothing useful at all. The sources overlap in some places and miss each other completely in others. So routing each question to the right source is now part of your harness.

Part of that routing is already in place. The `## Layout` section of `AGENTS.md` names a project skill for each kind of backend work: `minimal-api-endpoints` for endpoints, `service-layer` and `ef-core-postgres` for services, entities, and migrations, and `web-integration-tests` for integration tests. Those skills say how this repository does something. The documentation sources answer a different question: what a library offers in the version you have installed. The two can disagree. Microsoft Learn's Minimal API tutorial injects the DbContext straight into the endpoint handler, which is exactly what your `service-layer` skill forbids. So the routing also has to say which one wins.

There's a second problem too. It's about the file itself. Agents write padded, hedged prose, the kind that reads fine and instructs badly. `AGENTS.md` is loaded into context on every turn, so a wasted word is paid for again on every turn. The `writing-for-agents` skill teaches the agent to write a document whose reader is another agent, not a human.

So the prompt below asks for two things at once. Route the documentation sources, and write the routing in a form that costs almost nothing to load. The project skills stay where they are, under `## Layout`.

6.2.2 The prompt

```
Rework the "Up-to-date documentation" section of AGENTS.md.

You now have several documentation sources, and that section must tell you which
one to use when:

- .NET, C#, EF Core, ASP.NET Core: the `microsoft-docs` skill.
- Creating the Angular application: the `angular-new-app` skill.
- Before writing a larger piece of Angular code: read the `angular-developer`
  skill first.
- Any other detail, API signature, configuration option or how-to, for any
  library: the `find-docs` skill (Context7).

The "Layout" section already says which project skill to load for which backend
task. Do not repeat those skills in this section. Add one rule instead: when a
project skill and the documentation disagree, follow the project skill. It says
how this repository does something, and the documentation only verifies the API.

Read the `writing-for-agents` skill first and follow it. AGENTS.md is written for
an agent, not for a human, and it is loaded on every single turn.

Change only the "Up-to-date documentation" section. Leave the ".NET Project
conventions", "Layout" and "Quality gate" sections and every line in them
exactly as they are.
Do not make the file longer than it has to be.
```

What comes back should be a routing table, not an essay. Roughly this shape:

```
## Up-to-date documentation

Verify any API, project file property, configuration option or CLI flag in
the docs before using it:

- .NET, C#, EF Core, ASP.NET Core: the `microsoft-docs` skill.
- Creating the Angular app: the `angular-new-app` skill.
```

```
- Before writing a larger piece of Angular code: read the `angular-developer` skill.
- Any other library: the `find-docs` skill (Context7).

A project skill named under `## Layout` wins over the documentation. It says how
this repository does something, and the documentation only verifies the API.
```

Your wording will differ. What matters is that a reader can answer "which skill do I load for this question" right away, without a paragraph of preamble. Backend work hits two lines: the project skill from `## Layout` for the shape of the code, and `microsoft-docs` for the API it calls.

6.2.3 Read the diff, and read it for deletions

You just told an agent to rewrite a file that governs every future turn. Look at what it actually did:

```
git diff AGENTS.md
```

Two things to look for.

Did the other sections survive? The prompt names `.NET Project conventions`, `Layout`, and `Quality gate` by their headings and says to leave them exactly as they are. "Keep the existing conventions" is not precise enough on its own. An agent can read "conventions" as "the documentation rules" and then drop your `.slnx` rule and your C# 14 `extension(...)` rule. It may still report that it kept everything. A rewrite prompt without an explicit protection clause is a prompt that silently deletes things.

`## Layout` is the section most at risk here, because its bullets name skills too. An agent asked to route skills likes to pull them into the new section, or merge them into one list. List every line that names a skill:

```
grep -n "skill" AGENTS.md
```

The four backend skills still have to sit under `## Layout`, each with the task that loads it. The documentation section names `microsoft-docs`, the two Angular skills, and `find-docs`. A backend skill showing up there is a duplicate that the next rewrite will let drift apart from its original.

What else disappeared? Read every line the rewrite dropped, not only the ones the protection clause covers. One line that's likely to disappear is the note "there is no MCP server in this container, so use the CLI fallback." The `microsoft-docs` skill documents its own fallback, so dropping that note is defensible. But it was a line you wrote on purpose, and now it's not there. Decide whether you want it back, and add it yourself if you do. The same question applies to every other dropped line.

> **!** When an agent rewrites a file, `git diff` is not optional
>
> A summary of a rewrite is written by the thing that did the rewriting. It reports what the agent believes it did, not what landed on disk. The diff is the only account of the change that nobody had a reason to polish.

6.2.4 Pin a library id when the agent keeps guessing an old version

The routing table sorts questions by topic. You can route one level deeper and pin a single library. One library in this project has earned it: xUnit. In the quality gate part, the agent lost turns on xUnit v3's new test runner. A small model knows the old xUnit v2 idioms far better than v3. The v3 answers live on xunit.net, which Microsoft Learn does not cover.

Watch what happens when the agent asks Context7 for xUnit. Run the resolution step yourself:

```
npx ctx7@latest library xunit
```

You get four Context7 library ids for the same project. One is the documentation website. Another is a snapshot of the API reference pinned to one v3 release. The last two are mirrors of the GitHub repository. All four look plausible. Which one the agent picks is a fresh decision on every run. By now you know what to do with a decision like that: take it away. Try the website one, with a question you already know the answer to:

```
npx ctx7@latest docs /websites/xunit_net "what changed in the v3 test runner"
```

Read a screenful of what comes back. That's current v3 documentation, the material the model's training data does not have.

Now pin it. Add one line to the routing list in `AGENTS.md`, by hand, the same way you added the quality gate section there:

```
- xUnit v3: the `find-docs` skill with the Context7 library `/websites/xunit_net`.
```

The same move applies beyond this project. When the model keeps guessing an outdated version of one specific library, pin that library's documentation source by id in `AGENTS.md`. It's the same trick as pinning the folder layout in the first chapter of the course. It turns a choice the agent would otherwise make fresh every run into a choice made once, in the harness.

6.3 The Angular app

6.3.1 Pin the frontend layout

The backend got its folders from `AGENTS.md` and a skill. The frontend has neither yet, so the agent picks a structure on every run. One run puts every file flat in `src/app`. Another invents a `core/` folder and stops there. Each later page lands wherever that first run decided.

A structure exists already. The `angular-developer` skill recommends one in `references/naming-conventions.md`: backend access in `core/`, one folder per feature under `features/`, and reusable parts with no business logic in `shared/`. That file is one of about 40 references in the skill, and nothing sends the agent there. So do what you did for the backend. `AGENTS.md` names the folders, and the skill holds the details. Add these two bullets to the `## Layout` section of `~/work/AGENTS.md`, at the end:

```
- The frontend is one Angular application in `frontend/`. Inside
  `frontend/src/app`, the backend calls and their models live in `core/`, each
  page gets its own folder under `features/`, and components, pipes and
  directives with no business logic go in `shared/`.
- Before you create a file in `frontend/src/app`, read
  `references/naming-conventions.md` in the `angular-developer` skill and
  follow it.
```

The first bullet pins the folders that later commands and reviews depend on. The second one sends the agent to the file names and class names, which change with the Angular version and belong in the skill.

Commit the new bullets before you prompt:

```
git add AGENTS.md
git commit -m "Pin the frontend layout"
```

6.3.2 The prompt

This one prompt asks for the whole frontend, the service that talks to your backend, and a new gate stage. Most of it is ordinary scaffolding. Two bullets are not, so read them before you paste.

```
Add an Angular frontend to this repository.

- Create a single Angular application in a `frontend` folder. No library, no SSR,
  no PWA. Standalone and zoneless, with routing but without lazy loading.
- Styling is plain modern CSS using custom properties. Do not add Tailwind,
  Bootstrap or any other CSS framework.
- During development the frontend must reach the backend through a proxy on /api.
- One single Angular service holds every call to the backend: create, list all,
  get by id and delete for the Values entity, plus the add calculation.
- The reads that fetch stored Values (list all, get by id) use Angular's
  httpResource API. Do not write these reads as HttpClient methods that return
  an observable.
- Create and delete use HttpClient, because a resource is read-only. For the add
  calculation, pick whichever of the two fits the endpoint the backend has.
- No component calls HttpClient, httpResource or fetch itself. Components use the
  service.
- Test the service with Vitest, using Angular's HTTP testing support
  (provideHttpClientTesting and HttpTestingController).
- Add npm scripts for the tasks that matter: `start`, `build` and `test`.
- Use the latest stable versions of all npm packages.
- Do not create AI assistant config files (AGENTS.md, CLAUDE.md and similar).
  This repository already has one.
- Add a project skill in .agents/skills/api-access/SKILL.md. It documents this
  convention: where the backend calls live, which of them use httpResource and
  which use HttpClient, the development proxy, and when the file has to be
  touched.

Then add a stage to scripts/quality-gate.sh that builds the frontend for
production. Commit when the quality gate is green.
```

This is a hard run for a small model, harder than the quality gate was. `httpResource` is newer than the model's training data, so the agent cannot write a line of the service from memory. It has to read the `angular-developer` skill or ask Context7 first. Expect several turns, and expect at least one of them to produce observables anyway.

6.3.3 One bullet names the mechanism

Every prompt in this course states outcomes and lets the agent decide how. One bullet here breaks that rule on purpose, twice in two sentences. It names `httpResource` for the stored-data reads. Then it names the answer you refuse to accept: an `HttpClient` method returning an observable.

The reason is the model, not the requirement. The model has seen the `HttpClient` and observable pattern thousands of times, and `httpResource` almost never. So "read through the modern resource API" reads to it as "make a request somehow." What comes back then compiles and passes the gate. It shows the right numbers on the screen. So nothing you have built so far can tell you it went the wrong way.

Naming the wrong answer is the cheapest fix there is. It's also the kind of sentence you'll delete the day you move to a stronger model. The `agent_end` rule in the extension prompt is the same kind of thing: written for this model, not for the requirement.

> **If your agent builds more than you asked for**
>
> Some runs of this prompt produce more than scaffolding: a finished calculations page, sometimes a values page too, complete with tests. The next two chapters ask for exactly those pages. So the agent may reply "nothing to implement, everything already exists." That's not a problem, so keep the commit and paste the prompts anyway. Treat the chapters as review checklists for pages you already own. Every check in them still applies.

6.3.4 Review the skill the agent wrote

Everything except the skill bullet asks the agent for application code. The skill bullet asks it to write a piece of your harness. That's a different kind of work, and it deserves a different kind of review.

Open `.agents/skills/api-access/SKILL.md` once the run is done. A useful skill states four things.

The one place requests live. It names the path of the service file, inside `core/`, and states the rule that goes with it: components never make a request themselves.

Which mechanism goes with which direction. Reads go through `httpResource`. Create and delete go through `HttpClient`. A resource is read-only by design, so a write cannot go through one.

When the file has to change. The next agent can recognize these situations: a backend endpoint is added, renamed, or removed; a request or response shape changes; or the port behind the proxy changes.

The failure symptom. This is what a broken binding looks like from the outside. The frontend compiles without complaint. Then the request 404s at runtime, or it succeeds and the fields come back `undefined`, because the TypeScript type and the C# model disagree.

The failure symptom is what separates a skill from a README. A README documents a convention. A skill describes a situation the next agent will find itself in, and it tells the agent what to do about it. So it fixes the cause, instead of debugging the symptom for half an hour.

One way to write the two parts that matter:

```
---
name: api-access
description: How the Angular frontend talks to the .NET backend. Every request
  goes through the single API service; reads use httpResource, create and delete
  use HttpClient, and /api is proxied to the backend in development. Use when
  adding or changing a backend call, when a backend endpoint or model changes,
  or when a request fails at runtime.
---

## Failure symptom

The frontend compiles and still gets it wrong: the request 404s, or fields come
back undefined because the TypeScript type and the C# model disagree. Nothing
```

```
before the browser or the Playwright suite catches this.
```

Yours names the real path of your service file. The description above leaves that open, because the agent picked the path. Compare the rest with what your agent produced, and spend most of your attention on the `description`. Everything under the front matter is loaded only when the agent decides the skill is relevant. The agent reads the `description` on every turn and uses only that field to decide whether to load the skill. If the description says "documentation about the API service," the agent never loads the skill on the turn where it adds an endpoint. That one field is worth arguing about.

6.3.5 Check the wiring yourself

Start with the folders, because every later page builds on them:

```
find frontend/src/app -type d
```

No page exists yet, so `core/` is the folder that matters. The service and its test sit in it. A service in `src/app` itself means the agent skipped the layout bullets in `AGENTS.md`. Send it back to them before you prompt for the first page, because that page goes under `features/` and has to find the service where the layout says it is.

Three claims from the prompt are easy to verify and easy for an agent to get half right. Start with the proxy and the mechanism.

```
find frontend -name "proxy.conf.*" -not -path "*/node_modules/*"
grep -rn "httpResource" frontend/src
```

The first command finds the proxy config, because the scaffold decides where it sits: next to `angular.json` in some runs, under `src/` in others. Open the file it prints. It has to map `/api` to the port the backend actually listens on. So compare it against your launch settings instead of trusting the file.

The second grep is the decisive test in this chapter. It has to hit inside the service. And the hits have to be the stored-data reads: the list and the single value. A green gate proves the frontend compiles and the service tests pass. A page that shows the right numbers proves the requests arrive. Only this grep proves the reads went through the API you asked for instead of the one the model knows better.

If the grep comes back empty, the agent built the service out of `HttpClient` and observables and reported success. Send it back with the same two bullets, and add that the previous attempt used observables for the reads.

The third claim is that no component makes a request of its own. Look at where the requests come from:

```
grep -rn "HttpClient\|httpResource\|fetch(" frontend/src/app --include=*.ts
```

Two kinds of line are allowed in that output. Everything from the service file is allowed, because making requests is its job. `provideHttpClient()` in the application config is allowed too, because it's the registration the service needs. Anything else is a component talking to the backend on its own. The values page chapter turns this rough grep into a check worth keeping. For now, confirm that the requests all live in one file.

6.3.6 Look at it in the browser

```
cd frontend
npm start
```

Open `http://localhost:4200` on your machine. What you get is the page Angular scaffolds by default, since nothing has been built on top of it yet. That's the point: the toolchain runs, and the browser reaches it. Stop the server with `Ctrl+C` when you're done. A dev server left running holds port 4200, and the next run fails to bind it.

> **Getting to that port from your host**
>
> `compose.yaml` publishes port 4200, so `localhost:4200` in your host browser reaches the dev server inside the container. If the page doesn't load, the dev server is listening on the container's own loopback address only. Start it with `npm start -- --host 0.0.0.0` and try again.
> Port 4200 and the backend port are the only published ones. For anything else, the Ports panel in the attached VS Code window forwards a port for you. Forward a Port adds one by hand, when it doesn't appear there by itself.

6.4 The calculations page

6.4.1 The prompt

```
Add a calculations page to the frontend, with tests.

- The page shows a form with two number inputs. On submit it calls the backend
  add endpoint through the API service and displays the result.
- The result is displayed in green when it is >= 0 and in red when it is < 0.
- That red/green decision must live in an Angular pipe, not in the component.
- Test the pipe with Vitest.

Then add a stage to scripts/quality-gate.sh that runs the frontend Vitest tests
once, without watch mode. Commit when the quality gate is green.
```

6.4.2 Why the color rule has to live in a pipe

The third bullet looks like a style preference, but it's not. Testability is a design constraint, and you are the one who has to impose it.

Put the red/green decision in a template expression, and the only way to check it is a component test or a browser. Both are slow, and slow checks stay outside the per-turn loop. Put the same decision in a pipe, and it becomes a pure function that takes a number and returns a class name. It comes with a Vitest test that runs in milliseconds. That test fits inside the gate, so it runs after every single agent turn.

Where you put logic decides which loop can check it. That's the whole argument, and it applies far beyond Angular pipes.

6.4.3 Break it

You've added two gate stages in this step, the production build and Vitest. So far, you have only the agent's word that either one works. Prove it the same way you proved the analyzer and the migrations drift check: make the gate fail on purpose.

Find the pipe spec first, since the agent chose its name:

```
grep -rln "green" frontend/src --include=*.spec.ts
```

Open the spec file it prints and change one expected `green` to `red`. Then run the gate by hand:

```
bash scripts/quality-gate.sh
tail -1 quality-gate.log
```

The last line has to read `QUALITY GATE: RED`, and the failing Vitest assertion has to be visible in `quality-gate.log`. Revert your edit, run it again, and confirm the gate is green before you move on.

> ⚠ A Vitest stage without a no-watch flag hangs
>
> This break-it run matters more than the earlier ones. Vitest defaults to watch mode in some setups. A watch-mode stage inside the gate doesn't fail. It just waits. A hanging stage looks exactly like a slow stage until somebody sits there for ten minutes wondering why the gate never finishes. Reading the script tells you nothing here. The flag may be correct, and the installed Vitest version may still behave differently. Only running it does.

6.5 The values page

6.5.1 The prompt

```
Add a Values page to the frontend.

- One single page that can create a Values entry, list all entries, fetch one by
  id (showing both numbers and their sum), and delete one.
- Everything happens on that one page. No routing between sub-views.
- All backend access goes through the existing API service. Do not add an HTTP
  call in a component.
- Reuse the existing styling approach, plain CSS with custom properties.

Commit when the quality gate is green.
```

6.5.2 Check for bypasses

The third bullet is the one to watch. Some agents cannot decide which method of the service to call. Others find it quicker to write the request right where it's needed. Either way, the agent quietly injects `HttpClient` into the component instead. The page works and the gate stays green. The convention your `api-access` skill describes now protects nothing, because the requests no longer come from one place. Look for that directly.

Start by finding the service, since the agent chose where it lives:

```
grep -rln "httpResource" frontend/src/app --include=*.ts
```

Then list every place in the app that makes a request:

```
grep -rn "inject(HttpClient)\|httpResource\|fetch(" frontend/src/app --include=*.ts
```

Now look at what that second command deliberately does not search for. A bare `HttpClient` pattern would also match `provideHttpClient()` in `app.config.ts`. That call is not a bypass at all, it's the registration the service itself needs to work. So the pattern asks for `inject(HttpClient)`, which is how a component actually obtains one.

One legitimate hit is left, and it's the loudest one. The service file matches every pattern in the list. It should, because making requests is its whole job. Drop it from the results:

```
grep -rn "inject(HttpClient)\|httpResource\|fetch(" frontend/src/app --include=*.ts \
  | grep -v "<the service file your first grep found>"
```

Anything still printed is a component talking to the backend on its own. Read it, then make the agent move that call into the service.

A check that accuses correct code gets ignored within a week. An ignored check is worse than no check. It still costs time, and it teaches you that the gate can be skipped. Narrowing has a price, though, and you just paid it: a component that takes `HttpClient` through its constructor never says `inject(`. So this pattern never catches it. Widen it the day your agent starts writing constructors.

Every check you add to this harness goes through the same process. Ask what correct code it would accuse, and narrow it until the answer is nothing.

6.6 Playwright and tiers

6.6.1 The prompt

```
Add end-to-end tests with Playwright.

- Set up Playwright in the frontend project, latest stable version, Chromium only.
- One e2e test for the calculations page: enter two numbers, submit, check the
  result and that it is shown in the right color. Cover a negative result too.
- One e2e test for the values page: create a Values entry, see it in the list,
  open it by id and check the sum, then delete it.
- The tests need a running backend and a running frontend. Make that reproducible
  with npm scripts so one command runs the whole e2e suite. Name them
  `test:e2e` and `test:e2e:install`, and have the install script pass
  `--with-deps`.

Do NOT add Playwright to scripts/quality-gate.sh. These tests are far too slow to
run after every agent turn.

Keep your context small: read only the reference files you actually need.
Commit when the quality gate is green.
```

6.6.2 What webServer does for you

Read `frontend/playwright.config.ts` when the run finishes. The `webServer` block is the part that matters. Playwright starts the backend and the Angular dev server, then waits until both answer. After the suite runs, it shuts both down again. You never have to start two processes in two terminals, and nothing stays running afterward.

The `webServer` block is what makes an end-to-end suite reproducible. Suppose a suite passes only after a person starts the backend and the frontend by hand. Then it fails in CI, with an error nobody can reproduce locally.

6.6.3 Run the suite

```
cd frontend
npm run test:e2e:install   # once: Chromium plus system dependencies
npm run test:e2e
```

The install script runs once per container, because it downloads a browser and its system libraries. The suite itself prints something close to this:

```
Running 3 tests using 1 worker
  ok calculations page > adds two positive numbers and shows the sum in the success colour
  ok calculations page > shows a negative sum in the danger colour
  ok values page > creates a row, opens it by id, checks the sum and deletes it
  3 passed (9.0s)
```

Your run may have two tests or three, depending on how the agent split them. The names will be its own. They all have to pass.

Nine seconds sounds fast. Add the browser download, a `dotnet` build, two server startups, and a real database. The total is too slow for a check after every agent turn. That's why this suite stays out of the gate script.

6.6.4 Your gate now has tiers

Leaving Playwright out of `scripts/quality-gate.sh` was a design decision, and you just made it. Your checks now sit in tiers.

Loop	What runs	When
Per agent turn (`agent_end`)	format, analysis, unit and integration tests, migrations drift, frontend build, Vitest	automatically, every turn
On demand	Playwright end-to-end tests	when a human or the agent asks
Later	CI, deployment checks	on push

A check belongs in the per-turn loop if it's fast enough that nobody is ever tempted to skip it. Make the gate slow and someone skips it. A skipped check protects nothing.

6.6.5 What this step taught you

One gate, two toolchains. `scripts/quality-gate.sh` now drives `dotnet` and `npm` and still returns a single exit code.

Nothing cheap gates the contract. The TypeScript types in your API service and the C# on the other side describe the same requests twice, by hand. No tool compares the two. The Vitest tests for the service cannot see it either. `HttpTestingController` replaces the backend with one you invented in the test. So those tests prove the service builds the URL and the body you expect. But they run against a backend that always agrees. Rename a field in C# and every one of them still passes.

The values page end-to-end test is the only check in this project that drives the real chain, from the browser through Angular and .NET to Postgres. So the contract between your two toolchains is covered exactly once, in the slowest tier you own. That's a decision rather than an accident: catching contract drift costs a browser and two servers, and nothing cheaper can do it. Know which tier holds your only check for a given failure, because that's the tier you have to keep running.

Limit what the agent reads. The `angular-developer` skill ships 40 reference files. A small model that reads them all fills its context window with documentation and has nothing left for your task. That's why the Playwright prompt tells it to read only the reference files it needs.

Part 7

Skills

7.1 Write a skill

7.1.1 The procedure lives in your head

You built a Playwright end-to-end suite in the frontend part, and one npm script runs the whole thing. But that knowledge is not written down anywhere an agent will find it. You know which command starts the suite. You know that the backend and the Angular dev server both have to answer before the first test runs. You also know what to do when a port is still held by a server you forgot to stop. All of it lives in your head, and in a README nobody loads.

Ask an agent to "run the e2e tests" and it improvises. It starts a server here and forgets to stop one there. A stray `dotnet run` holds port 5074 for the rest of the afternoon.

A skill is where a procedure like that belongs. You have installed ten of them by now, and the chapter "The cutoff problem" showed how they load: the description sits in the agent's context on every turn, and the body only when a task matches. So the body can be as long and as specific as the procedure really is.

The agent has also written one for you already. The Angular prompt in the frontend part asked for an `api-access` skill as one bullet in a long list. Now write one deliberately, as the whole task.

7.1.2 The prompt

```
Write a project skill that describes how to run the end-to-end tests: starting the
backend, starting the frontend, running the Playwright suite, and stopping
everything again afterward.

Read the `writing-for-agents` skill first and follow it. This document is written
for an agent, not for a human.

Put it in .agents/skills/e2e-tests/SKILL.md. Commit when the quality gate is green.
```

The second paragraph is what changes the output. Drop it and you get a README: background, then the command you need buried in the middle. The prompt sends the agent to `writing-for-agents` and tells it who the reader is. That turns the output into a document an agent can follow.

7.1.3 Read it like a harness engineer

Open `.agents/skills/e2e-tests/SKILL.md` and check it for four things. The examples below show what good looks like. Yours will read differently, so use them as the standard, not as the expected answer.

A description that names the situations. The `description` is the only part of a skill that sits in the agent's context on every turn. It decides whether the body is ever opened. A description that says what the skill is fires when the agent happens to think of it. A description that names the situations fires when those situations happen. A good one ends with something like "and when a run fails because a URL is already used by a leftover server". That is a second situation, and a less obvious one. Read your description and ask which situations would make an agent open this file. If the only answer is "when someone asks to run the e2e tests", it is too narrow.

Steps with completion criteria. Not "run the tests", but "done when the exit code is 0 and the list reporter shows every test passed". An agent needs to know when it is finished. If you do not tell it, it decides for itself and usually stops too early.

A failure section. An agent can often improvise a success path but not a recovery path. That's why failure modes are the part of a skill it cannot invent. A good one documents the "port already used" case together with its fix, `pkill -f 'ng serve'`. This is the item most likely to be missing from your file. Your agent wrote the skill without ever watching the suite fail, so it had nothing to document. An absent failure section is a finding. Write the section yourself from what you know, or ask for it in a follow-up prompt.

Rationale for the constraints. Whatever constraints your Playwright config carries, the skill has to say why they are there. A skill that only repeats a setting adds nothing the config file does not already say. If

your config pins one worker, the skill owes you the reason. A good one reads: "Sequential by design (`workers: 1`): the tests share the real database." Yours may run more workers and carry different constraints. The same question applies to each of them. Without the why, a future agent helpfully turns parallelism back on and the tests fight over the same rows. You then spend an afternoon on intermittent failures.

> Compare the e2e skill with the api-access skill
>
> Open `.agents/skills/api-access/SKILL.md`, the one the Angular prompt asked for. One file describes a procedure, and the other describes a convention. Both are useful for the same reason. They say when they apply and how the thing they cover fails.

Fix what the review finds before you move on. A skill you have read and corrected is one you can start calling on purpose.

7.1.4 Force it and watch

A skill normally loads when the agent decides it is relevant. Forcing it is how you find out whether the one you just wrote works. In `pi`, type:

```
/skill:e2e-tests
```

Whatever you type after the command name is handed to the skill as the request:

```
/skill:e2e-tests only the values spec
```

You watch the agent follow your own steps instead of guessing whether the description would have fired. Where it improvises, the skill is missing a step. Fix the file, run `/reload` so the session picks up the change, and force it again. The forced skill also turns a procedure into a command you can call by name instead of a document you have to reread.

7.1.5 What this step taught you

Skills are how a harness remembers procedures. `AGENTS.md` loads on every turn, so it can only hold what is always true. A skill loads when it becomes relevant, so it can carry detail that would be waste in `AGENTS.md`: the exact commands, the failure modes, and the reason each constraint exists.

The description decides whether the skill is read. A perfect procedure behind a vague description does nothing, so spend your editing time on the one line that sits in context on every turn.

The harness now writes parts of itself. You used an agent, guided by a skill about writing for agents, to write a skill for agents. The agent produced a draft, and you made it complete by checking it against the four things a skill needs: triggers in the description, completion criteria in the steps, a failure section, and rationale for the constraints.

The frontend still has no gate. The backend has had a formatter and a static analyzer since the quality gate part, and the frontend has neither. That is the next thing to fix, and then you can measure what the whole gate costs you.

Part 8

The last stage

8.1 Lint the frontend

The backend has had a formatter and a static analyzer since you built the quality gate. The frontend has had neither. The Angular app, the calculations page, the values page, and the Playwright suite are all written by the agent. A build and a few Vitest runs are the only checks on them. Nothing in `scripts/quality-gate.sh` checks how that TypeScript looks or which patterns it uses.

This chapter adds ESLint and Prettier to the frontend. It also adds a lint and format stage to `scripts/quality-gate.sh`. That completes the harness. The next chapter measures what the full gate costs to run after every agent turn.

8.1.1 The prompt

Start `pi` and paste this:

```
Add linting and formatting to the Angular frontend.

Follow https://www.angulararchitects.io/en/blog/best-practices-prettier-eslint/
but keep it simple: this is a teaching project, not a production app.

- ESLint for TypeScript and Angular templates, Prettier for formatting.
- The two must not fight each other about formatting.
- npm scripts to check and to fix.
- Add a stage to scripts/quality-gate.sh that runs the lint and the format check.

Then prove the gate works: introduce a formatting problem and an ESLint violation
in one frontend file, run the gate, confirm it turns red, then fix them and get it
back to green.

Keep your context small: read only what you actually need.
Commit when the quality gate is green.
```

8.1.2 Two bullets prevent two failures

"The two must not fight each other about formatting." ESLint and Prettier overlap. Both have opinions about where a line breaks and how many spaces sit in front of it. Wire them together naively and you get rules that undo each other. Prettier writes a line one way. An ESLint layout rule reports it as wrong. You fix it for ESLint, and the next format run puts it back. That gate can never turn green. The usual fix is a config that turns off ESLint's layout rules and leaves formatting to Prettier, but check what your agent actually did. Naming the conflict in the prompt is much cheaper than debugging it afterward.

"Then prove the gate works." You have already broken the analyzer gate, the migrations drift check, and the database container, all on purpose. A gate you've never seen fail is a gate you're guessing about. So the prompt tells the agent to do the breaking. The next chapter asks you to repeat it yourself: the agent claiming red is not the same as you watching red.

8.1.3 Expect a big diff either way

Turning Prettier on for the first time reformats the whole frontend. The diff touches nearly every file the agent has ever written there. ESLint finds problems in code that was written before it existed: unused imports and `any` where a type belonged. Some runs split the result into two commits, configuration first and cleanup second. Others land everything in one. Either way, the reformat noise and the real fixes arrive together. That is exactly why you read the commit list before the diff.

Check what actually landed:

```
git log --oneline -5
git show --stat HEAD
```

Look for a commit whose stat block lists most of the frontend, with only one or two changed lines per file. That commit is the Prettier sweep, and the pattern is normal, not a sign that something went wrong. What you

don't want is a single commit that mixes the reformat with a behavior change. Then nobody can review either half. If the agent produced that, say so and ask it to split them.

Once the gate is green, go read what the agent added to `scripts/quality-gate.sh`.

8.2 Check, never fix

The agent says the frontend lint stage is in. Go look.

8.2.1 Count the stages

`scripts/quality-gate.sh` most likely has a helper that runs one stage and prints a header. Open the script and find out what yours is called before you grep for anything. A common name is `run_stage`, but your agent may have named it `run_step` or plain `step` instead. Some agents skip the helper for one stage and write it inline. No single grep pattern fits every script.

The count you are after is in the log, not in the script. Every stage prints a header when it runs, so run the gate and count the headers:

```
bash scripts/quality-gate.sh
grep -c "^=== " quality-gate.log
```

Adjust the pattern to match your script's headers. Run `tail -50` of the log to see what they look like. The exact count is the agent's choice, not a target. One run bundles ESLint and Prettier into a single stage. Another gives `npm ci` a stage of its own. Match each header in the log to a check you added on purpose. Look for a stage you cannot explain, or a check that never got a stage, and chase that down before you go on.

`dotnet` and `npm` are two different ecosystems, but they run behind the same entry point: one script. The extension calls that script. So does your shell, and one day a CI job will too. None of them needs to know anything about either toolchain.

8.2.2 The npm scripts come in pairs

Open `frontend/package.json` and read the `scripts` block. The lint and format entries come in pairs, one that reports and one that repairs. They look something like this, even if your names differ:

```
"lint": "eslint .",
"lint:fix": "eslint . --fix",
"format": "prettier --write .",
"format:check": "prettier --check ."
```

Now check which half of each pair the gate calls. It has to be the checking half.

A gate that silently fixes your code hides the very thing it was built to reveal. It also makes the agent's work look cleaner than it was. The agent writes badly formatted TypeScript. The gate reformats it and reports green. You never learn that the agent needs a stronger instruction. The evidence is gone, erased by the thing that was supposed to collect it.

This is the same reason the backend stage runs `dotnet format --verify-no-changes` instead of plain `dotnet format`. Report, don't repair. If your agent wired `format` or `lint:fix` into the script, change it now.

8.2.3 Settling the indentation conflict

The backend indents with 4 spaces. Prettier defaults to 2 for TypeScript. The conflict only bites once two toolchains share a repository. Both `.editorconfig` and `.prettierrc` set indentation rules, and they can disagree.

There are at least two reasonable answers:

- A scoped section in `.editorconfig`, something like `[frontend/**]`, that gives the frontend its own 2-space rule.
- Letting `.prettierrc` control everything under `frontend/` and leaving `.editorconfig` for the backend.

Your agent may have picked either. What you're checking is that somebody decided, not which one they decided. The failure mode is the case where nobody decided. Two tools quietly overwrite each other's indentation, and one file flips back and forth between 2 and 4 spaces depending on which tool ran last. The gate catches that

as a format check that fails forever, no matter how often you run the fixer. Knowing why is much faster than bisecting your config files.

8.2.4 Break it yourself

Don't just believe what the agent says. Mangle one line:

```
# put some random extra spaces into a frontend .ts file, then:
bash scripts/quality-gate.sh
tail -1 quality-gate.log
```

The log looks roughly like this, with your own file name:

```
=== Stage: frontend lint (ESLint) + format check (Prettier) ===
[warn] src/app/calculations/result-color.pipe.spec.ts
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
RESULT: FAIL (exit code 1)

QUALITY GATE: RED
```

The stage header and the Prettier wording depend on how your agent wrote the script. `QUALITY GATE: RED` and exit code 1 are the stable part. Those are what the extension and any future CI job read. Then `git checkout` the file and run the gate again. Confirm it comes back GREEN.

8.2.5 One file, two verdicts

Read the rest of the gate log, not just the last line. The Vitest stage passed for the same file, in the same run, while the format check failed on it. The code was correct and badly formatted at the same time. The harness said so, in one sentence each.

The gate carries this many stages because different stages answer different questions. Vitest answers "does the logic work." Prettier answers "is it formatted." ESLint checks whether the code follows our conventions. None of the three can replace the others. You need all of them, and the next chapter is about what that costs.

8.3 Count the cost

The harness is finished, and it has a real price. Every stage in `scripts/quality-gate.sh` runs after every agent turn. That means you pay for all of them every time the agent stops to think. Time a full run:

```
time bash scripts/quality-gate.sh
```

Write the number down. That number is the tax on every turn from here on.

8.3.1 The honest questions

Now ask the questions you would ask about anyone else's build:

- **Which stages could run only when the files they care about changed?** Nothing in the frontend moved, so ESLint had nothing to do. The script could ask git which files changed since the last green run. Then it could skip the stages those files cannot affect.
- **Does `npm ci` need to run on every gate invocation?** It reinstalls the whole dependency tree from `package-lock.json`. That lock file changes maybe once a week. The install only matters when that file changed.
- **Which stage is slowest, and is it worth the time?** Not every slow stage is wasted time. The integration tests are slow because they talk to a real database. That is the whole point of them. Cut the stage that is slow and catches little.
- **Which stages should move to CI?** Playwright already showed you the tiering pattern: fast checks belong in the per-turn loop, slow checks run on demand only, and everything else waits for a push. Any check you skip belongs in a slower tier.

A harness you dread is a harness you disable. Here is the usual path on a real team. The pre-commit hook gets slower. Somebody discovers `--no-verify`. Within a month the gate protects nothing, because nobody runs it. A fast gate is a gate people keep running.

8.3.2 What this step taught you

A gate checks and never fixes. `--check` and `--verify-no-changes` exist so that a gate reports instead of repairing. An auto-fixing gate destroys the evidence you built it to collect. It tells you the code was fine, when really the gate just fixed it.

Coverage is a series of deliberate choices. Look at what the stages actually cover now:

Question	Stage
Does it compile?	`dotnet build`
Does it follow our conventions?	analyzers, ESLint
Is it formatted?	`dotnet format`, Prettier
Does the logic work?	xUnit, Vitest
Does it work against a real database?	integration tests
Are generated artifacts current?	migrations drift check
Does the whole stack work?	Playwright, on demand

Nobody planned that table in advance, including you. Each row got added the moment your harness let something through: a build that passed with sloppy formatting, an entity that changed without a migration, a test suite that never touched Postgres. Real harnesses get built one leak at a time.

8.3.3 What you actually did here

You spent six steps on this project and wrote almost no application code. The agent wrote the app. You built the machine that decides whether the app is acceptable. You broke every part of that machine on purpose, before you trusted any of it.

You now have a harness you can defend. You know what each stage catches, and you know what the whole run costs.

8.3.4 Where to take it next

You have a working harness in front of you. Here are three next steps.

Take the slow tier to CI. Playwright is the only check you own that drives the real chain from the browser to Postgres. That makes it the only check that would catch your frontend and backend disagreeing about a field name. Right now it runs when somebody remembers. Put it in a GitHub Actions workflow that runs on every push. At the same time, take the slowest stage from your timing run out of the per-turn loop. You have already made the tiering decision once, so the second time goes faster.

Gate a generated artifact in your own project. The migrations drift check works because you can ask EF Core whether the generated file still matches its source. Go find the equivalent in the next project you touch. Something produced each one: a lock file, a snapshot test, generated TypeScript, a compiled schema. You can ask whether it is still current, and that answer belongs in the gate.

Build the same harness around a stronger model. Everything here ran on GPT-6 Luna, a low-cost model, on purpose. A small model trips the gates often enough that you can watch each one catch a failure. At home, point a stronger agent at your next project and give it the same treatment: an `AGENTS.md` with the rules, a skill for each procedure, one gate script, and a hook that runs it after every turn. A better model writes better code. It does not tell you whether the code is acceptable. That part is still your job.

A Troubleshooting

Something will misbehave. When it does, look for the symptom in the left column before you read your own code. Almost everything that breaks in this course is a harness problem, not a code problem: an extension never loaded, a stopped container, a trust prompt nobody accepted. The application code is usually fine. You fix the harness around it.

Symptom	Cause and fix
`pi` reports an authentication error	`auth.json` next to `compose.yaml` is missing, is malformed, or holds the activity code instead of your key. A key starts with `nvk-`. Check the JSON shape against the example in the "Your toolkit" chapter, and open the activity page again to read your key. The page always hands back the same one, so a key that stops working means your teacher closed the activity.
`pi` fails with a confusing configuration error on first start	`auth.json` did not exist when the container first started, so Docker created a directory under that name. On your host, in the folder that holds `compose.yaml`: `docker compose down`, delete the `auth.json` directory, run `cp auth.json.example auth.json` and paste your key in, then `docker compose up -d`.
`pi` will not start, and node reports an error about a bootloader script	VS Code's JavaScript auto-attach sets `NODE_OPTIONS=--require .../bootloader.js` in the terminals it opens, and `pi` fails to start with it. Run `unset NODE_OPTIONS` in that terminal, then start `pi` again. Auto-attach is a setting in your own VS Code rather than something in the container, so turn it off with the command **Debug: Toggle Auto Attach** if it keeps coming back.
Agent does not know about `microsoft-docs`	The project is not trusted. Restart `pi` and accept the trust prompt, or start it with `pi -a`.
`ls ~/skill-source` prints nothing	The `skill-source/` folder was not next to `compose.yaml` when the container started, so Docker mounted an empty directory instead. Put the folder from your teacher back next to `compose.yaml`, then run `docker compose up -d` again
`dotnet test` rejects a flag such as `--nologo`	xUnit v3 runs on the Microsoft Testing Platform, which refuses some flags the older VSTest runner accepted. Current packages work this way by design, so your code is not the problem. Let the agent read the error and adjust the test invocation in the gate script.
`dotnet format` fails on files under the migrations folder	The EF Core migration generator writes files the formatter and analyzers dislike, starting with a byte order mark. Add a scoped `.editorconfig` section for the migrations folder. Check that the scope covers that folder and nothing more. See the "The harness pays off" chapter.

Symptom	Cause and fix
Build warning `MSB3277` (conflicting assembly versions) in the test project	The Npgsql provider's dependency ranges can pull different versions into the API and test projects. `TreatWarningsAsErrors` turns that warning into a red gate. Let the agent align the package versions across both projects.
The gate never runs by itself	The extension has to exist before the session starts, so the session that wrote it never loaded it. Restart `pi` or run `/reload`. Then check the timestamp on `quality-gate.log` after the next turn.
The gate still never runs after a restart	Project-local extensions only load in a trusted project, and only from `.pi/extensions/`. Accept the trust prompt or start with `pi -a`. Then check where the file landed.
The gate goes red and the agent stops anyway	The extension is listening on `agent_settled` instead of `agent_end`. A message on `agent_settled` never starts a new turn. See the "Automate the gate" chapter.
`npx @microsoft/learn-cli` is slow the first time	The first call downloads the package. Later calls use the cache.
Agent runs out of context while reading `quality-gate.log`	The log gets long. Tell the agent to read only the tail, or make the gate script log a summary.
A run went badly wrong	Run `git checkout .` and `git clean -fd`, then rephrase the prompt. Resetting is cheaper than arguing with the agent.
Gate red with `NpgsqlException`	The `db` container is not running. Run `docker compose start db` on your host.
`dotnet ef` not found	Rebuild the container with `docker compose build dev`. Or let the agent add a local tool manifest and run `dotnet tool restore`.
Integration tests fail only in a full run	They assume an empty database, and they share that database with the app. See the "Integration tests" chapter.
`pi` prints nothing for several minutes	A request to the shared endpoint was lost. Abort and send the same prompt again. It usually succeeds. Run only one `pi` session at a time, because concurrent sessions lose requests more often.
`This extension ctx is stale after session replacement or reload`, and `pi` dies	The gate extension's `agent_end` handler returns before it has sent its message, so the gate talks to a session that has already been replaced. The handler has to await the gate and send the message before returning. See the "Automate the gate" chapter.
The agent keeps retrying a gate it cannot fix	Nothing limits the number of retries, and the gate stays red for a reason outside the code. Limit the consecutive retries. See the "Close the loop" chapter.
`Author identity unknown` when the agent commits	You never set `git config user.name` and `git config user.email`. See the "Git is your undo button" chapter.
Playwright says "port is already used"	A previous run ended without releasing the port. Run `pkill -f 'ng serve'` and `pkill -f 'backend/Calculator.Api'`.
Agent uses up its context on documentation	The `angular-developer` skill ships 40 reference files. Tell the agent to read only what it needs.

Symptom	Cause and fix
`~/work` is empty and everything is gone	Somebody ran `docker compose down -v`, which deletes the `work_data` volume that holds it. Plain `docker compose down` and a rebuild both keep it. See the note in the "Git is your undo button" chapter.
The API responds inside the container, but your browser cannot reach port 5074	Check whether the API listens only on the container's loopback address. Ask the agent to set `applicationUrl` to `http://0.0.0.0:5074` in the `launchSettings.json` profile used by `dotnet run`. Restart the API and retry from your browser. `compose.yaml` already publishes port 5074.
Browser cannot reach the app's port	`compose.yaml` publishes only ports 4200 and 5074. For any other port, use the VS Code **Ports** panel in the attached window to forward it.
`git checkout .` did not undo the run	`git checkout .` only reverts tracked files. The agent's new files stay until you run `git clean -fd`.